

Debremarkos University

Burie Campus

Computer Science Department

- *Web Programming-* CoSc 3082
- *Chapter Four: JavaScript(JS)*

Outlines: Chapter 4: Client Side Scripting (JavaScript)

Chapter 4: Client Side Scripting (JavaScript) (8hr)

4.1. Introduction to JavaScript

4.2. JavaScript Basic

4.2.1. JavaScript Syntax

4.2.2. Attaching JavaScript to HTML (External, Embedded, Inline)

4.3. JavaScript Comments

4.4. Basic JavaScript Input Output

4.5. JavaScript Data Types and Variables

4.5.1. JavaScript Data types

4.5.2. Variable declaration in JavaScript

4.5.3. Data Type Conversion

4.6. Arithmetic and Logical Operators in JavaScript

4.7. Control Structures (Conditional and Looping Statements)

4.8. Array in JavaScript

4.9. JavaScript Functions

4.10. JavaScript DOM (Document object Model)

4.10.1. Accessing HTML elements in JavaScript

4.10.2. CSS in JavaScript

4.10.3. Events in JavaScript

4.10.4. Handling Exception in JavaScript

4.11. Form Processing using JavaScript

4.12. JavaScript BOM (Browser Object Model)

4.12.1. JavaScript Window

4.12.2. JavaScript Location

4.12.3. JavaScript Location

4.12.4. JavaScript Cookies

Scripting language

- A **scripting language** is a type of programming language **used to write scripts—short programs that automate tasks, control software, or add functionality to websites or applications.**
- Web scripting languages are used to **add interactivity, dynamic content, and automation** to websites. They can run on the **client side (frontend)** or **server side (backend)**.

Type	Description	Examples
Client-side	Run in the browser (frontend)	JavaScript, TypeScript
Server-side	Run on the web server (backend)	PHP, Node.js (JavaScript), Python (Django, Flask), Ruby (Rails), ASP.NET

Html vs CSS vs JS

Feature	HTML (HyperText Markup Language)	CSS (Cascading Style Sheets)	JavaScript (JS)
Purpose	Structure & content of a webpage	Styling and layout	Interactivity and dynamic behavior
What it does	Defines elements like headings, links, images, forms	Applies colors, fonts, spacing, positions	Responds to user actions, manipulates DOM, fetches data
Syntax Example	<code><h1>Hello World</h1></code>	<code>h1 { color: red; }</code>	<code>document.querySelector('h1').style.color = 'blue';</code>
Runs where?	Browser (static content)	Browser (applied with HTML)	Browser (interpreted by JS engine)
Can it make a website alone?	Yes, but plain and unstyled	No, needs HTML to apply styles	No, needs HTML/CSS for structure & style
File extension	.html	.css	.js

JavaScript

1. **HTML** to define the content and structure of web pages.
 2. **CSS** to specify the layout of web pages.
 3. **JavaScript** to program the behaviour of web pages.
- JavaScript was originally known as **LiveScript** by **Netscape** in the mid 1990s. It was later renamed to **JavaScript** in 1995.
 - JavaScript is the most popular and **widely used client-side scripting language**.
 - **Client-side scripting** refers to **scripts that run within your web browser**.
 - **JavaScript** is designed to add interactivity and dynamic effects to the web pages by manipulating the content returned from a web server.
 - In addition, client-side scripts can **control browser actions** associated with a Web page.
 - Client-side scripts are almost written in the **Javascript** language to control browser's actions.
 - Client-side scripting can make Web pages more **dynamic** and more **responsive**

Tasks performed by client-side scripts

- Checking **correctness** of user input
- **Monitoring** user events and **specifying reactions**
- **Replacing** and **updating** parts of a page
- Changing the **style** and **position** of displayed elements **dynamically**
- **Modifying** a page in **response** to **events**
- Getting browser **information**
- Making the Web page **different** depending on the browser and browser features
- **Generating HTML** code for parts of the page

What JavaScript can do?

1. JavaScript Can Change HTML Content

- One of many HTML methods is **getElementById()**.
- This example uses the method to "find" an HTML element (with id="demo"), and changes the element content (**innerHTML**) to "Hello JavaScript":

```
<body>
<h1>What Can JavaScript Do?</h1>
<p id="demo">JavaScript can change HTML content.</p>
<button type="button"
onclick="document.getElementById('demo').innerHTML = 'Hello JavaScript!'">
Click Me!</button>
</body>
```

- To access an HTML element, JavaScript can use the **document.getElementById(id)** method.
- The **id** attribute defines the HTML element.
- The **innerHTML** property defines the HTML content:

What JavaScript can do?

2. JavaScript Can Change HTML Styles (CSS)

Changing the style of an HTML element, is a variant of changing an HTML attribute:

```
<body>
<h1>What Can JavaScript Do?</h1>
<p id="demo">JavaScript can change the style of an HTML element.</p>
<button type="button" onclick="myFunction()">Click Me!</button>
<script>
function myFunction() {
    var x = document.getElementById("demo");
    x.style.fontSize = "25px";
    x.style.color = "red";
}
</script>
</body>
```

3. JavaScript Can Validate Data

➤ JavaScript is often used to validate input:

4. JavaScript Can Change HTML Attributes

Popular Libraries and Frameworks

- **React:** A library for building user interfaces, particularly single-page applications.
- **Angular:** A platform and framework for building single-page client applications using HTML and TypeScript.
- **Node.js:** A runtime environment that allows execution of JavaScript on the server side.

Advantages of JS

- **Web Development:** JavaScript is a fundamental technology of the World Wide Web, alongside HTML and CSS. It is used to enhance user interfaces and improve user experiences by making web pages more interactive and dynamic.
- **Server-Side Development:** With the advent of Node.js, JavaScript is also used for server-side development, allowing developers to use JavaScript for full-stack development.
- **Mobile App Development:** Frameworks like React Native allow developers to create mobile applications using JavaScript.
- **Game Development:** JavaScript, along with HTML5, is used to develop browser-based games.

Syntax & Where to Place JS

- JavaScript can be placed in the **<body>** and the **<head>** sections of an **HTML page** or placed as **external**.
- The **<script>** tag is used to define a client-side script (**JavaScript**).
- The **<script>** element either contains **scripting statements**, or it points to an **external script** file through the **src** attribute.
- Example : **JavaScript in <body>**

- Placing the JavaScript code directly inside an **HTML tag** using the special tag attributes such as **onclick**, **onmouseover**, **onkeypress**, **onload**, etc.

Code

```
<html>
  <head>
    <title>HTML script Tag</title>
  </head>
  <body>
    <script type="text/javascript">
      // Java Script Code Here
    </script>
  </body>
</html>
```

JavaScript in <head>

- The **<script>** tag can be used to define a function under the <head> tag of HTML.
- The JavaScript code will lie under <script> tag in a function which can be invoked inside the <body> tag of HTML.

```
<head>
<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<h1>My Web Page</h1>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
```

❖ *JavaScript in <body>*

- In this example, a JavaScript function is placed in the <body> section of an HTML page.
- The function is invoked (called) when a button is clicked:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Demo JavaScript in Body</h2>
```

```
<p id="demo">A Paragraph.</p>
```

```
<button type="button" onclick="myFunction()">Try it</button>
```

```
<script>
```

```
function myFunction() {
```

```
    document.getElementById("demo").innerHTML = "Paragraph changed.";
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

External JavaScript

- Scripts can also be placed in external files.
- External scripts are practical when the same code is used in many different web pages.
- JavaScript files have the **file extension .js**.
- To use an external script, put the name of the script file in the **src** (source) attribute of the `<script>` tag:

Syntax: `<script src="PathToJS">`
`</script>`

To add several script files to one page - use several script tags:

```
<script src="myScript1.js"></script>
```

```
<script src="myScript2.js"></script>
```

External References

An external script can be referenced in 3 different ways:

- With a full URL (a full web address)
- With a file path (like `/js/`)
- Without any path

Code

```
<html>
  <head>
    <title>HTML script Tag</title>
  </head>
  <body>
    <script src="PathToJS">
    </script>
  </body>
</html>
```

External JavaScript

message.html

```
<head>
  <script src="externalJS.js"></script>
</head>
<body>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
```

message.js

```
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
```

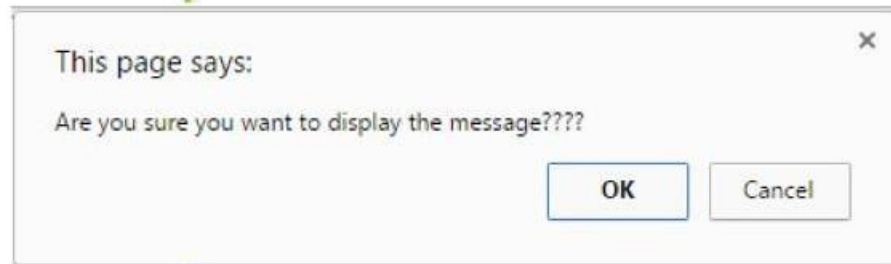
External JavaScript (Example)

message.js

```
function myAlert(msg) {  
    if(confirm("Are you sure you want to display the message????")) {  
        alert(msg);  
    }  
    else {  
        alert("Message not Displayed as User Canceled Operation");  
    }  
}
```

myHtml.html

```
<html>  
  <head>  
    <script src="message.js"></script>  
  </head>  
  <body>  
    <script> myAlert("Hello World"); </script>  
  </body>  
</html>
```



JavaScript comments

- JavaScript comments are used to add explanations, notes, or reminders to your code.
- Comments are ignored by the JavaScript engine during execution.
- There are two types of comments in JavaScript:
 - single-line comments
 - multi-line comments.

Single-Line Comments

- Single-line comments start with `//`.
- Everything following `//` on that line will be considered a comment.
- Example:

```
// This is a single-line comment
```

```
let x = 5; // This is an inline comment
```

Multi-Line Comments

- Multi-line comments start with `/*` and end with `*/`. They can span multiple lines.

```
/* This is a multi-line comment.  
It can span multiple lines.*/
```

Input Output in JavaScript

- When you are working with a programming language, you want to execute some logic and then see the output, or display the final output.
- In JavaScript, to display output, there are 4 simple and different ways
 - Using an **Alert Box**
 - By logging on to the **Console**
 - Using **innerHTML** property (or the **textContent** property)
 - Using **document.write()** method

window.alert()

- You will find many websites where messages are displayed in alert boxes.
- The message can be some welcome message, or some alert message if you are trying to submit some incorrect value in any form, etc.
- Using the `document.alert()` or `alert()` method, it's very easy to create an alert popup on a webpage and display any message in it.

➤ You can use an alert box to display data:

➤ Example

```
<body>
```

```
<h1>My First Web Page</h1>
```

```
<p>My first paragraph.</p>
```

```
<script>
```

```
var x = 10;
```

```
var y = 20;
```

```
var sum = x + y;
```

```
alert(sum); // Outputs: 30
```

```
</script>
```

```
</body>
```

document.write()

- `document.write()` is a method using which you can write something on a webpage directly using javascript.
- Note: `document.write()` method overwrites all of the things inside an HTML document,

➤ For testing purposes, it is convenient to use **`document.write()`**:

Example

```
<html>
<body>
<h1>My First Web Page</h1>
<p>My first paragraph.</p>
<script>
var x = 10; var y = 20;
var sum = x + y;
document.write(sum);
</script>
</body>
</html>
```

Using innerHTML

- JavaScript lets you add content to any HTML element by using the innerHTML property.
- To do that first you need to provide a specific ID to the HTML element that you want to access by the JavaScript code.
- To access an HTML element JavaScript uses document.getElementById(id) method, where id is the value of the id attribute of the HTML tag
- The **innerHTML** property defines the HTML content.

```
<body>

  <p id="greet"></p>
  <p id="result"></p>
  <script>

    // Writing text string inside an element
    document.getElementById("greet").innerHTML = "Hello World!";
```

```
// Writing a variable value inside an element

    var x = 10;
    var y = 20;
    var sum = x + y;
    document.getElementById("result").innerHTML = sum;
  </script>
</body>
```

Writing Output to Browser Console

- You can easily outputs a message or writes data to the browser console using the `console.log()` method.
- This is a simple, but very powerful method for generating detailed output.
- Here's an example:
- ExampleTry this code »

```
// Printing a simple text message
```

```
console.log("Hello World!"); // Prints: Hello World!
```

```
// Printing a variable value
```

```
var x = 10;
```

```
var y = 20;
```

```
var sum = x + y;
```

```
console.log(sum); // Prints: 30
```

Tip: To access your web browser's console, **first press F12 key** on the keyboard to **open the developer tools** then **click on the console tab**.

Input in JavaScript

- In JavaScript, input and output operations are essential for interacting with users.
- Below are some of the basic methods to handle input in JavaScript.
 - **prompt()**
 - **HTML Form Input**

prompt()

- The `prompt()` method displays a dialog box that prompts the user for input. It returns the input value as a string.
- If the user clicks "Cancel," it returns null.

```
<script>  
  let userInput = prompt("Please enter your name:");  
  document.write("User's name is:", userInput);  
</script>
```

Input in JavaScript

■ HTML Form Input

- You can also get input from users through HTML forms and JavaScript.

```
<body>
<form id="myForm">
  Name: <input type="text" id="nameInput">
  <input type="button" value="Submit" onclick="displayInput()">
</form>
<script>
function displayInput() {
  let name = document.getElementById("nameInput").value;
  document.write("User's name is:", name);
}
</script>
</body>
```

JavaScript Data types

- Data types basically specify what kind of data can be stored in a variable and manipulated within a program.
- There are six basic data types in JavaScript which can be divided into three main categories:
 1. primitive (or primary): String, Number, and Boolean etc
 2. composite (or reference): Object, Array, and Function
 3. special data types: Undefined and Null are special data types.

```
var length = 16;           // Number
var lastName = "Abebe";    // String
var cars = ["Saab", "Volvo", "BMW"]; // Array
var x = {firstName:"kebede", lastName:"Abebe"}; // Object
```

String Data type in JavaScript

- Whenever we write a character or a sequence of characters or numbers or special characters, or anything **inside single or double quotes** then it becomes a String.
- For example "computer science", "1234", "STU\$%12", etc.
- `var department = "computer science"; // Using double quotes`
`var department = ' computer science ';` // Using single quotes
- `var answer = "It's alright"; // Single quote inside double quotes`

JavaScript Numbers

- JavaScript has only one type of numbers.
- Numbers can be written with, or without decimals:
- Example
`var x1 = 34.00; // Written with decimals`
`var x2 = 34; // Written without decimals`

JavaScript Booleans

Booleans can only have two values: true or false.

Example

```
var x = true;
```

```
var y = false;
```

Reading Ass: array,object and other datatypes

undefined, null, **empty value**, and Symbol in JavaScript:

- **undefined**: A variable that has been **declared but not assigned a value**.

```
let a;
```

```
console.log(a); // undefined
```

- Undefined: Used by JavaScript by default to indicate **absence of assignment**.

- **null**: An explicit assignment to represent **no value or intentional emptiness**.

```
let b = null;
```

```
console.log(b); // null // Used when a developer intentionally clears a variable.
```

- **Empty Value**: A variable that holds an empty string (""), empty array ([]), or empty object ({}).

- Type: Depends on the value (string, array, object, etc.)

```
let str = "";  
let arr = [];  
let obj = {};
```

Concept	Description	Type	Assigned by	Used for
undefined	No value assigned	undefined	JavaScript	Uninitialized vars, missing args
null	Explicit "no value"	object	Developer	Clear/reset value
Empty value	Initialized but empty ("", [])	varies	Developer	Empty data structures
Symbol	Unique identifier	symbol	Developer	Hidden/unique object keys

- **Symbol**: A Symbol is a unique and immutable primitive value introduced in ES6 (ECMAScript 2015).
- It is often used to create unique object keys.
 - `const sym = Symbol("description");`
 - `console.log(sym);` // Output: Symbol(description)

JavaScript statements

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called **statements**.
- A **JavaScript program** is a list of programming **statements**.
- In HTML, JavaScript programs are executed by the web browser.

➤ JavaScript statements are composed of:

Values, Operators, Expressions, Keywords, and Comments.

JavaScript Values

- The JavaScript syntax defines two types of values: **Fixed values and variable values**.
- Fixed values are called **literals** (e.g 10.50 , 1001).
 - Variable values are called **variables**.(e.g)

JavaScript Variables

- In JavaScript variables declared using three different keywords: var, let, and const.

1. var: Variables declared with **var** are **function-scoped or globally-scoped**, meaning they are **accessible within the function they are defined in or globally** if defined outside of a function.

- Variables declared with var can be reassigned and redeclared within their scope.

- Example:

✓ **var** x = 5;
var y = 6;
var z = x + y;

```
let num = 100;  
let num = 200; //syntax error  
  
var num = 100;  
var num = 200; //Ok
```

2. let: Variables declared with let **are block-scoped**, meaning they are **accessible within the block** (e.g., a pair of curly braces {}) they are defined in.

- Variables declared with **let** **can be reassigned**, **but cannot be redeclared within the same scope**.

```
let y = 5;  
let y = 10; // This will throw a SyntaxError: Identifier 'y' has already been declared  
y = 15; // This is valid, y is now 15
```

JavaScript Variables

3. const: is used to declare variables with constant values.

- Variables declared with **const** are block-scoped, just like **let**.
- **Constant Value:** Variables declared with **const** must be initialized at the time of declaration and **cannot be reassigned**.
- Variables declared with **const** cannot be reassigned, but their properties can be modified if the variable holds an object or an array.
- Example 1 : `const e = 25;`

`// e = 30; // TypeError: Assignment to constant variable`

Syntax: **const** var cannot be reassigned but properties can be modified if the variable holds an object or an array.

```
const obj = { key: 'value' };
```

```
obj.key = 'new value'; // This is allowed
```

```
const person = { name: "John" };
```

```
person.name = "Jane"; // This is valid, the property can be modified
```

```
person = { name: "Jane" }; // This will throw a TypeError: Assignment to constant variable.
```

```
const num = 100;  
num = 200; //error  
const name; //error  
name = "Steve";
```

Scope of a variable

- The scope of a variable is the area of the program where it can be used. It can be used in the entire program or only in a specific part of the program.
- The scope of a variable is determined by the location where it is declared.
- It can be declared in the global scope, in a function, in a loop, etc.
 - **Scopes are of 2 types:**
 - Global Scope - The variable is accessible in the entire program
 - Local Scope - The variable is accessible only in the function where it is declared

Example:

```
var user = "abebe"; // global scope
function sayHello() {
  console.log(user);
}
sayHello();
```

JavaScript Identifiers

- There are some rules while declaring a JavaScript variable (also known as identifiers).
 - All JavaScript **variables** must be **identified** with **unique names**.
 - These unique names are called **identifiers**.
 - Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).
 - The general rules for constructing names for variables (unique identifiers) are:
 - Names can contain letters, digits, underscores, and dollar signs.
 - Names must begin with a letter.
 - Names can also begin with \$.
 - Names are case sensitive (y and Y are different variables).
 - Reserved words (like JavaScript keywords) cannot be used as names.

JavaScript Type Conversion

- In JavaScript, **type conversion** means changing a value from one data type to another. It can happen **implicitly (type coercion)** or **explicitly (manual conversion)**.
- **Implicit Type Conversion** :JavaScript **automatically** converts types during operations
 - `console.log('5' + 3);` `// "53"` (number 3 converted to string)
 - `console.log('5' - 3);` `// 2` (string '5' converted to number)
 - `console.log(true + 1);` `// 2` (true converted to 1)
 - `console.log(false + 'x');` `// "falsex"`

Operation	Example	Result	Explanation
String + Number	"5" + 2	"52"	Number 2 is converted to a string and concatenated.
Number + Boolean	5 + true	6	Boolean true becomes 1 (false → 0).
String - Number	"10" - 5	5	String "10" is converted to a number.
Loose Equality (==)	"5" == 5	true	String "5" is converted to a number.
Strict Equality (===)	"5" === 5	false	No conversion (different types).

Converting Strings to Numbers

■ Explicit Conversion:

- You can explicitly convert data types using **built-in JavaScript functions and constructors**.
- The global **method Number()** converts a variable (or a value) into a number.
 - A numeric string (like "3.14") converts to a number (like 3.14).
 - An empty string (like "") converts to 0.
 - A non numeric string (like "John") converts to **NaN (Not a Number)**.

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML =
```

```
Number("3.14") + "<br>" +
```

```
Number("  ") + "<br>" +
```

```
Number("") + "<br>" +
```

```
Number("99 88") + "<br>" +
```

```
Number("John") + "<br>";
```

```
</script>
```

output

3.14

0

0

NaN

NaN

JavaScript Type Conversion

// Special cases:

```
Number(true);    // 1
Number(false);   // 0
Number(null);    // 0
Number(undefined); // NaN
```

null vs undefined in Conversions

```
Number(null);    // 0
Number(undefined); // NaN
String(null);    // "null"
String(undefined); // "undefined"
Boolean(null);   // false
Boolean(undefined); // false
```

To boolean

```
Boolean(1);      // true
Boolean(0);      // false
Boolean("hi");   // true
Boolean("");     // false
Boolean([]);     // true (empty array is truthy!)
Boolean({});     // true (empty object is truthy!)
Boolean(null);   // false
Boolean(undefined); // false
```

- **String():** Converts a value to a string.
 - Example : let x = String(42); // x will be "42"
 - **Boolean():** Converts a value to a boolean.
 - let x = Boolean(0); // x will be false
 - let y = Boolean(1); // y will be true
-

Converting Dates to Numbers

- The global method Number() can be used to convert dates to numbers.
- d = new Date();
- Number(d) // returns 1404568027739

○ **Converting Dates to Strings**

- The global method String() can convert dates to strings.
- String(Date()) // returns "Thu Jul 17 2014 15:38:19 GMT+0200 (W. Europe Daylight Time)"

Converting Booleans to Numbers

The global method Number() can also convert booleans to numbers.

- Number(false) // returns 0
- Number(true) // returns 1

Converting Booleans to Strings

The global method String() can convert booleans to strings.

- String(false) // returns "false"
- String(true) // returns "true"

JavaScript Operators

- JavaScript operators are symbols that are used to perform operations on operands.
- For **example**: `var sum=10+20;` Here, `+` is the arithmetic operator and `=` is the assignment operator.
- There are following types of operators in JavaScript.
 - ✓ Arithmetic Operators (`+`, `-`, `*`, `/`, `%`, `++`, `--`)
 - ✓ Comparison (Relational) Operators
 - ✓ Bitwise Operators
 - ✓ Logical Operators
 - ✓ Assignment Operators
 - ✓ Special Operators

JavaScript Arithmetic Operators

Operator	Name	Example
+	Addition	$3 + 4 // 7$
-	Subtraction	$5 - 3 // 2$
*	Multiplication	$2 * 3 // 6$
/	Division	$4 / 2 // 2$
%	Remainder	$5 \% 2 // 1$
++	Increment (increments by 1)	$++5$ or $5++ // 6$
--	Decrement (decrements by 1)	$--4$ or $4-- // 3$
**	Exponentiation (Power)	$4 ** 2 // 16$

JavaScript Comparison Operators

- The JavaScript comparison operator compares the two operands.
- The comparison operators are as follows:

Operator	Description	Example
<code>==</code>	Is equal to	<code>10==20 = false</code>
<code>===</code>	Identical (equal and of same type)	<code>10===20 = false</code>
<code>!=</code>	Not equal to	<code>10!=20 = true</code>
<code>!==</code>	Not Identical	<code>20!==20 = false</code>
<code>></code>	Greater than	<code>20>10 = true</code>
<code>>=</code>	Greater than or equal to	<code>20>=10 = true</code>
<code><</code>	Less than	<code>20<10 = false</code>
<code><=</code>	Less than or equal to	<code>20<=10 = false</code>

JavaScript Logical Operators

➤ The following operators are known as JavaScript logical operators.

Operator	Description	Example
&&	Logical AND	<code>(10==20 && 20==33) = false</code>
	Logical OR	<code>(10==20 20==33) = false</code>
!	Logical Not	<code>!(10==20) = true</code>

JavaScript Assignment Operators

➤ The following operators are known as JavaScript assignment operators.

Operator	Description	Example
=	Assign	10+10 = 20
+=	Add and assign	var a=10; a+=20; Now a = 30
-=	Subtract and assign	var a=20; a-=10; Now a = 10
=	Multiply and assign	var a=10; a=20; Now a = 200
/=	Divide and assign	var a=10; a/=2; Now a = 5
%=	Modulus and assign	var a=10; a%=2; Now a = 0

Conditional statements

- A conditional statement is a statement that is used to make a decision. When the condition is given is true then the code is executed.
- JavaScript has the following conditional statements:
- **if statement** - if statement is used to execute a block of code only when a given condition is true.
- **else statement** - else statement is used to execute a block of code only when a given condition is false.
- **else if statement** - else-if statement provides a new condition to be checked if the previous condition was false to execute a new block of code.
- **switch statement** - switch statement is a shorter and cleaner way to use when there are many conditions to be checked and it works the same as an if-else statement.

JavaScript if statement

- if statement is the most **basic conditional statement**. It executes a block of code only when the given condition is true.
- if the condition is false then execution leaves the block and jumps to the next section.
- Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Example:

```
let x = 10;  
if (x == 10) {  
    console.log("Statement is true, x = 10");  
}
```

JavaScript else statement

- The else statement is **executes a code block when the condition in the if statement is false.**
- else statement can't be used alone it must be used after the if statement.
- Example:

```
if (condition) {  
    // executes this block if the condition is true  
} else {  
    // executes this block if the condition is false  
}
```

```
var a = 15;  
if (a == 10) {  
    console.log("True, a = 10");  
}  
else {  
    console.log("False, a is not equal to 10");  
}
```

else if statement JavaScript

- The else if statement is used to execute a block of code when the condition in if statement is false.
- The else if statement is similar to the else statement but it is used to check multiple conditions.

```
if (condition) {  
    // executes this block if the condition is true }  
else if (condition) {  
    // executes this block if the condition is false  
}  
... (more else if blocks)  
else {  
    // executes this block if the condition is false }
```

```
let today = new Date().getDay();  
if (today == 0) {  
    console.log("Sunday!"); }  
else if (today == 1) {  
    console.log("Monday!"); }  
else if (today == 2) {  
    console.log("Tuesday!"); }  
else if (today == 3) {  
    console.log("Wednesday!"); }  
else if (today == 4) {  
    console.log("Thursday!"); }  
else if (today == 5) {  
    console.log("Friday!"); }  
else {  
    console.log("Saturday!");}
```

Switch statement javascript

- The switch statement is a conditional statement (just like if..else) that allows the programmer to execute different blocks of code depending on the value of a variable.

```
switch (expression/variable) {  
  case expression1:  
    // execute if expression1 = expression  
    break;  
  case expression2:  
    // execute if expression2 = expression  
    break;  
  ...  
  default:  
    // execute if no match  
}
```

❖ This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, the default code block is executed.
- The break keyword is used to end each case block. If you do not use the break keyword, then execution will continue to the next case block.

```
<html> <body> <h2>JavaScript switch</h2> <p id="demo"> </p> <script> let day;
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
    day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case 6:
    day = "Saturday"; }
document.getElementById("demo").innerHTML = "Today is " + day;
</script></body></html>
```

Loops

Loops are used to execute the same block of code again and again, as long as a certain condition is met.

for loop

Use when you know how many repetitions you want to do

syntax

```
for(initialize ; condition ; increment) { ...  
}
```

example

```
for(x=0;x<10;x++) {  
    // Code Here  
}
```

while loop

Loops through block of code while condition is true

syntax

```
while(condition) { ... }
```

example

```
while (x<10) {  
    //Code Here  
}
```

do while loop

Execute block at least once then repeat while condition is true

syntax

```
do{ ... } while (condition);
```

example

```
do{  
    // Code Here  
} while (x<10)
```

for...in and for...of

- **for...in** — Iterates over keys (property names) Used to loop through enumerable properties (keys) of an object (including inherited ones).
- **Best For:** Objects and Accessing keys in key-value pairs
- Syntax

```
for (var propertyName in object) {  
    // do something  
}
```

```
<script>  
var person = {  
  name: "Abebe", age: 27, city: "Burie"  
};  
for (var prop in person) {  
  console.log(prop + ": " + person[prop]);  
}</script>
```

```
name: Abebe  
age: 27  
city: Burie
```

```
var fruits =  
["Apple", "Banana", "Mango", "Papaya"];  
for (var i in fruits) {  
  console.log(i + ": " + fruits[i]);  
}
```

```
0: Apple  
1: Banana  
2: Mango  
3: Papaya
```

- **for...of** — Iterates over values:
 - Used to loop through iterable values like
arrays, strings, sets, maps, etc.

```
for (var propertyName of object) {  
  // do something  
}
```

```
var fruits = ["Apple", "Banana", "Mango", "Papaya"];  
for (var fruit of fruits) {  
  console.log(fruit);  
}
```

array in JavaScript

- An **array** in JavaScript is a special variable that can store **multiple values** in a **single variable**.
- It's an ordered list of elements, where each element is accessible by its index.

Creating Arrays

- Array Literal (Most Common)

```
const fruits = ['Apple', 'Banana', 'Orange'];
```

- Array Constructor

```
const numbers = new Array(1, 2, 3);
```

- Empty Array

```
const emptyArray = []; const anotherEmpty = new Array();
```

Accessing Elements

- Arrays are zero-indexed:

```
const colors = ['Red', 'Green', 'Blue'];
```

```
console.log(colors[0]); // 'Red'
```

```
console.log(colors[2]); // 'Blue'
```

```
console.log(colors[3]); // undefined (out of bounds)
```

Arrays are Objects

- Arrays are a special type of objects.
- The **typeof** operator in JavaScript returns "object" for arrays.
- But, JavaScript arrays are best described as arrays.
- Arrays use **numbers** to access its "elements".
- In this example, person[0] returns John:

body>

<p id="demo"></p>

<script>

const person = ["Abebe", "kebede", 23];

document.getElementById("demo").innerHTML =

person[0];

</script></body></html>

- Objects use **names** to access its "members".
- In this example, person.firstName returns John:

body>

<p id="demo"></p>

<script>

const person = {firstName:"John", lastName:"Zeki",
age:26};

document.getElementById("demo").innerHTML =
person.firstName;

</script></body>

Array Properties and Methods

- The real strength of JavaScript arrays are the built-in array properties and methods:

`cars.length` // Returns the number of elements

`cars.sort()` // Sorts the array

The length Property

- The length property of an array returns the length of an array (the number of array elements).

```
<p id="demo"></p>
```

```
<script>
```

```
<script>
```

```
var fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];
```

```
document.write(fruits.length); // Outputs: 5
```

```
</script></script></body>
```

- **Accessing the First Array Element**

✓ `const` fruits = ["Banana", "Orange", "Apple", "Mango"];

`let` fruit = fruits[0];

- **Accessing the Last Array Element**

```
<p id="demo"></p>
<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits[fruits.length-1];
</script></body></html>
```

Looping Array Elements

- One way to loop through an array, is using a for loop:

```
<p id="demo"></p>
<script>
var fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];
for(var i = 0; i < fruits.length; i++){
    document.write(fruits[i] + "<br>"); // Print array element
}
</script></body></html>
```

```
var fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];
// Iterates over array elements
for(var fruit of fruits) {
    document.write(fruit + "<br>"); // Print array element
}
```

- **The `forEach()`** method is a built-in array method in JavaScript that allows you to iterate over the elements of an array and execute a provided function for each element.

```
array.forEach(function(currentValue, index, arr) {  
    // Perform some operation on the currentValue  
});
```

- **The `forEach()`** method takes a callback function as an argument, which is executed for each element in the array.
- The callback function can accept up to three parameters:
 - **currentValue:** The current element being processed in the array.
 - **index (optional):** The index of the current element in the array.
 - **arr (optional):** The original array being iterated over.

- **Example:**

Example 1: Basic Usage

```
const numbers = [1, 2, 3, 4, 5];  
numbers.forEach(function(number)  
{  
  console.log(number);});
```

Example 2: Using Index

javascript

```
const fruits = ['apple', 'banana', 'cherry'];  
fruits.forEach(function(fruit, index) {  
  console.log(`${index}: ${fruit}`); });
```

Adding Array Elements

- to **add a new element at the beginning of an array** use the **unshift() method**.
- **Example:**
 - ```
var colors = ["Red", "Green", "Blue"];
colors.unshift("Yellow"); document.write(colors); // Prints:
Yellow,Red,Green,Blue document.write(colors.length); //
Prints: 4
```
- To **add a new element at the end of an array**, simply use the **push() method**,
- **Example:**

```
var colors = ["Red", "Green", "Blue"];
colors.push("Yellow");
document.write(colors); // Prints: Red,Green,Blue,Yellow
document.write(colors.length); // Prints: 4
```
- **add multiple elements at once** using the **push()** and **unshift()** methods,

```
var colors = ["Red", "Green", "Blue"];
colors.push("Pink", "Voilet");
colors.unshift("Yellow", "Grey");
document.write(colors); // Prints: Yellow,Grey,Red,Green,Blue,Pink,Voilet
document.write(colors.length); // Prints: 7
```

## Array Methods

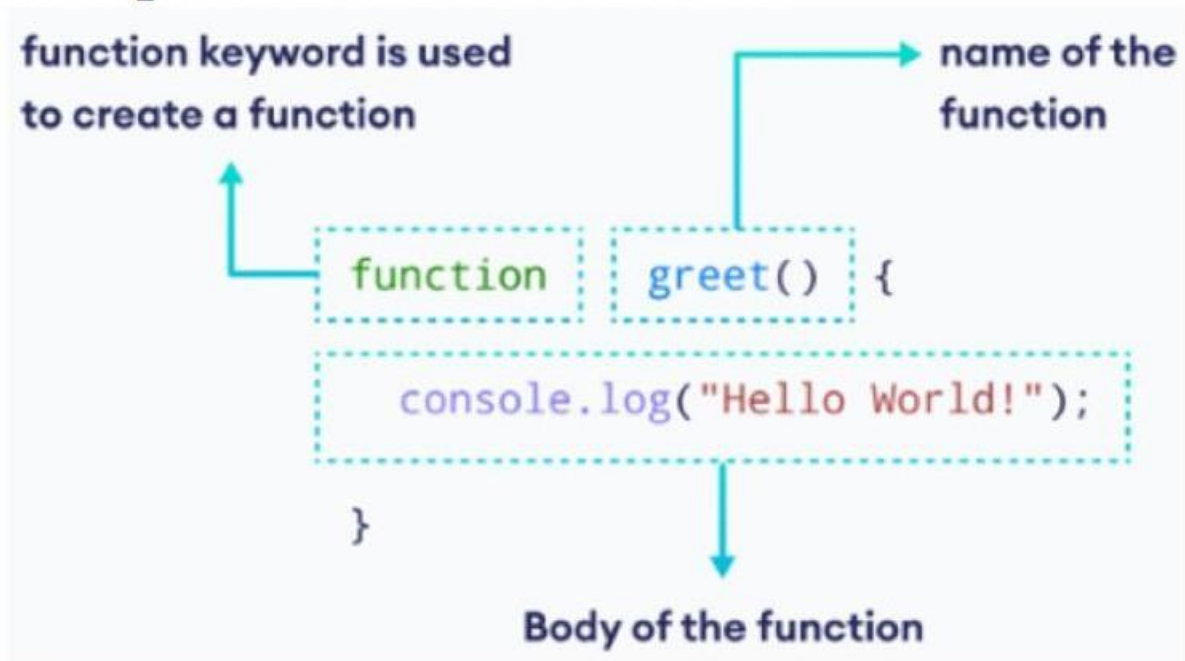
| Method                               | Description                                                                                                             |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#"><u>concat()</u></a>      | Returns a new array comprised of this array joined with other array(s) and/or value(s).                                 |
| <a href="#"><u>every()</u></a>       | Returns true if every element in this array satisfies the provided testing function.                                    |
| <a href="#"><u>filter()</u></a>      | Creates a new array with all of the elements of this array for which the provided filtering function returns true.      |
| <a href="#"><u>forEach()</u></a>     | Calls a function for each element in the array.                                                                         |
| <a href="#"><u>indexOf()</u></a>     | Returns the first (least) index of an element within the array equal to the specified value, or -1 if none is found.    |
| <a href="#"><u>join()</u></a>        | Joins all elements of an array into a string.                                                                           |
| <a href="#"><u>pop()</u></a>         | Removes the last element from an array and returns that element.                                                        |
| <a href="#"><u>push()</u></a>        | Adds one or more elements to the end of an array and returns the new length of the array.                               |
| <a href="#"><u>reduceRight()</u></a> | Apply a function simultaneously against two values of the array (from right-to-left) as to reduce it to a single value. |
| <a href="#"><u>reverse()</u></a>     | Reverses the order of the elements of an array -- the first becomes the last, and the last becomes the first.           |
| <a href="#"><u>shift()</u></a>       | Removes the first element from an array and returns that element.                                                       |
| <a href="#"><u>slice()</u></a>       | Extracts a section of an array and returns a new array.                                                                 |
| <a href="#"><u>toSource()</u></a>    | Represents the source code of an object                                                                                 |
| <a href="#"><u>splice()</u></a>      | Adds and/or removes elements from an array.                                                                             |
| <a href="#"><u>toString()</u></a>    | Returns a string representing the array and its elements.                                                               |

# JS Functions

- A JavaScript function is a **block of code** designed to perform a particular task.
- A JavaScript function is **executed** when "**something**" **invokes** it.
- A JavaScript function is defined with the **function** keyword, **followed by a name, followed by parentheses ()**.
- The **parentheses** may **include parameter** names **separated** by **commas**: (parameter1, parameter2, ...)
- The **code to be executed**, by the function, is placed inside **curly brackets**.
- **Javascript Function Declaration**
- A function declaration is a way to declare a function in javascript. It is a way to tell javascript that you want to create a function and to give it a name.
- the **function is declared** by using the **function** keyword.

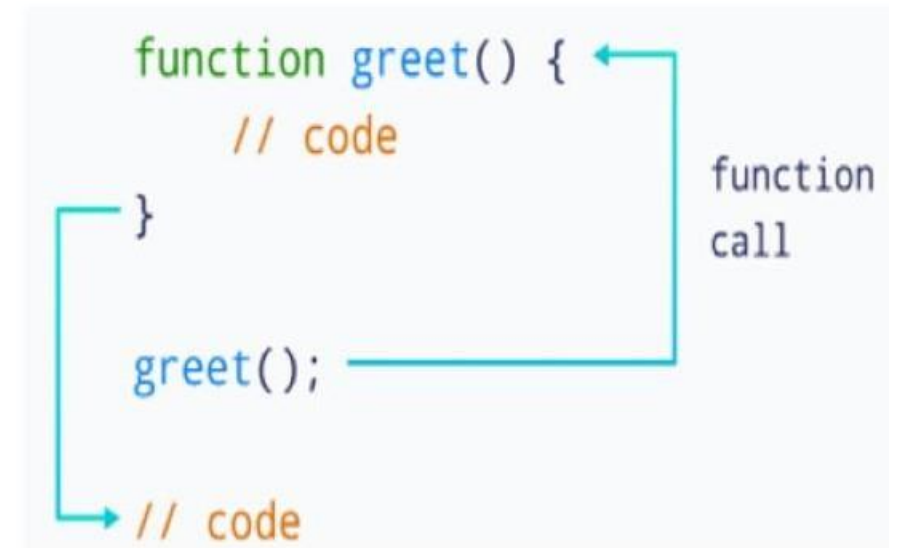
```
function functionName(parameters) {
 // code to be executed
}
```

## ➤ Example: function declaration



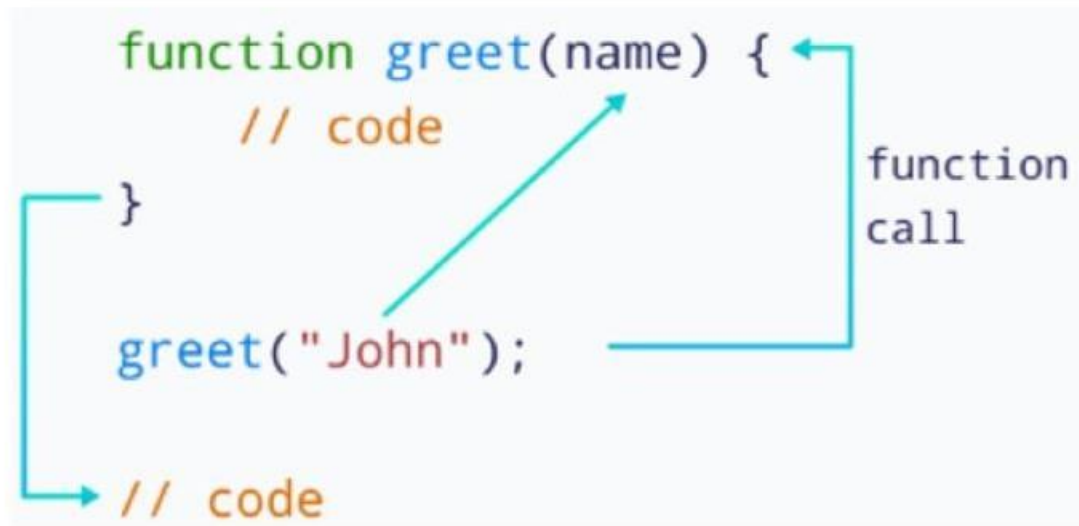
## call a function in javascript

- To call a function, you need to use the **function name followed by a parenthesis** and the arguments that you want to pass to the function.



# Javascript function parameter

- The actual values that are passed in the function are known as **arguments** or Arguments are values you pass to the function when you call it. while the **variable** that is declared in the function is known as a **parameter**.



- Here, `name` is a function parameter, which acts as a **placeholder to store the function argument**.
- we passed `"John"` as an **argument to the greet() function**.

What happens when you provide more or fewer parameters to the function than the defined arguments?

- **When a number of arguments are less than parameters then the rest parameters will be undefined**
- **when the number of arguments are more than parameters then extra arguments will be ignored**

## Example:

---

- `function fullName(firstname, lastname) {`
- `console.log(firstname + " " + lastname);`
- `}`
- `fullName("Abebe", "Ayana"); // Abebe Ayana`
- `fullName("yibeltal", "Ayana", "awol"); // Abebe Ayana`
- `fullName("Abebe"); // Abebe undefined`

# The return Statement

---

- We can return a value from a JavaScript function using the **return** statement.

```
function multiply(a, b) {
 return a * b;
}
// storing return value in a variable
var value = multiply(4, 6);
console.log(value);
```

# Function Expression

- A function expression in JavaScript is a function that is stored as a value, and can be assigned to a variable or passed as an argument to another function.
- In JavaScript, a function expression is a way to store functions in variables.
- Once function expression has been stored in a variable, the variable can be used as a function:

```
const functionName = function(parameters) {
 // code to be executed
};
```

```
<body>
 <script>
 // Function Declaration
 function getSum(num1, num2) {
 var total = num1 + num2;
 return total;
 }
 document.write(getSum(28, 2) + "
"); // Prints:20
```

```
// Function Expression
var Sum = function(num1, num2) {
 var total = num1 + num2;
 return total;
}
document.write(Sum(41, 6)); //
Prints: 10
</script>
</body>
```

# Anonymous function

---

A function expression is a way to define a function within an expression. It can be anonymous or named.

// Anonymous function expression ...function expression without name

```
const add = function(a, b) {
 return a + b;
};
```

// Named function expression

```
const multiply = function multiply(a, b) { return a * b;};
```

# Arrow Functions

- The function expression is followed by an arrow ( $\Rightarrow$ ).
- The arrow function expression is followed by an expression, which is returned by the arrow function.

## Arrow Function Syntax

```
let myFunction = (arg1, arg2, ...argN) => {
 statement(s)
}
```

Here,

- **myFunction** is the name of the function.
- **arg1, arg2, ...argN** are the function arguments.
- **statement(s)** is the function body.

If the body has single statement or expression, you can write the arrow function as:

```
let myFunction = (arg1, arg2, ...argN) => expression
```

## Example1: Arrow Function With One Argument

If a function has only one argument, you can omit the parentheses.

```
const square = x => x * x;
// use the arrow function to square a
number
```

```
console.log(square(5));
```

## Example :2

```
const sayHello = () => "Hello, World!";
// call the arrow function and print its return value
console.log(sayHello());
// Output: Hello, World!
```

# Using function expression: vs Using an arrow function:

---

// Anonymous function expression

```
const add = function(a, b) {
 return a + b;
};
```

// Named function expression

```
const multiply = function multiply(a, b) {
 return a * b;};
```

```
const add = (a, b) =>
```

```
{ return a + b;
};
```

// For single expression, implicit return

```
const multiply = (a, b) => a * b;
```

# JavaScript Objects

---

- An object is just a special kind of data, with properties and methods.
- Accessing Object Properties
  - Properties are the values associated with an object.
  - The syntax for accessing the property of an object is below  
*objectName.propertyName*
  - This example uses the length property of the Javascript's inbuilt object(String) to find the length of a string:  
*var message="Hello World!";*  
*var x=message.length;*

# Javascript object

- A **Javascript object** is a collection of properties and methods.
- It is a way to group data together and store it in a structured way.
- The data in objects are stored in **key-value pairs**. Where each key-value pair is called a property.

The syntax of JavaScript object is:

```
const objectName = {
 key1: value1,
 key2: value2,
 ...,
 keyN: valueN
};
```

Here,

objectName - Name of the object.

key1: value1 - The first key-value pair.

key2: value2 - The second key-value pair.

keyN: valueN - The Nth key-value pair

```
// student object
const student = {
 firstName: "Abebe",
 rollNo: 32 };
```

```
console.log(student);
// Output: { firstName: 'Abebe', rollNo: 32 }
```

# Accessing Object's Properties

There are two ways to access the properties of an object.

○ Using the dot operator and Using the bracket operator.

## dot operator

- Using the dot operator, you can access the properties of an object by writing the object name followed by a dot and the property name.
- `objectName.propertyName`
- The dot operator is most commonly used to access the properties of an object.

```
let car = {
 name: "Ford",
 color: "Black",
}
// accessing the property
console.log(car.name);
console.log(car.color);
```

## bracket operator

- Another way to access the properties of an object is using the bracket operator.
  - `objectName[propertyName]`
- It accesses the properties of an object by writing the object name followed by the bracket and the property name as a string.

```
let car = {
 name: "Ford",
 color: "Black"
}
// accessing the property
console.log(car["name"]);
console.log(car["color"]);
```

## Adding properties to an Object

- You can add a new property to an existing object simply by object name followed by a dot and the property name and assigning the value to the property.

```
let car = {
 name: "Ford",
 color: "Black"
}
console.log(car);
// adding a new property
car.model = "Mustang";
console.log(car.model);
```

## Deleting properties from an Object

- You can delete a property from an object by using the **delete operator**.

```
let car = {
 name: "Ford",
 color: "Black",
 model: "Avenger"
}
console.log(car.model);

// deleting a property
delete car.model;
console.log(car.model);
```

# Looping through an Object

---

- Here we have discussed 4 different ways to loop through an object.
- **1. Using `for...in` loop**
- The `for...in` loop access the properties of an object by iterating through the object.

```
<body>
 <script>
 var person = {
 name: "Peter",
 age: 28,
 gender: "Male"
 };

 // Iterating over object properties
 for(var i in person) {
 document.write(person[i] + "
"); // Prints: name, age
and gender
 }
 </script>
</body>
```

# Document Object Model (DOM)

- Every web page resides inside a browser window which can be considered as an object.
- window object represents the window or frame that displays the document and is the global object in client side programming for JavaScript.
- All the client side objects are connected to the window object.
- Window object – Top of the hierarchy. It is the outmost element of the object hierarchy
- A Document object represents the HTML document that is displayed in that window.
- The Document object has various properties that refer to other objects which allow access to and modification of document content.
- Form object – Everything enclosed in the <form>...</form> tags sets the form object.
- Form control elements – The form object contains all the elements defined for that object such as text fields, buttons, radio buttons, and checkboxes.

# Document Object Model (DOM)

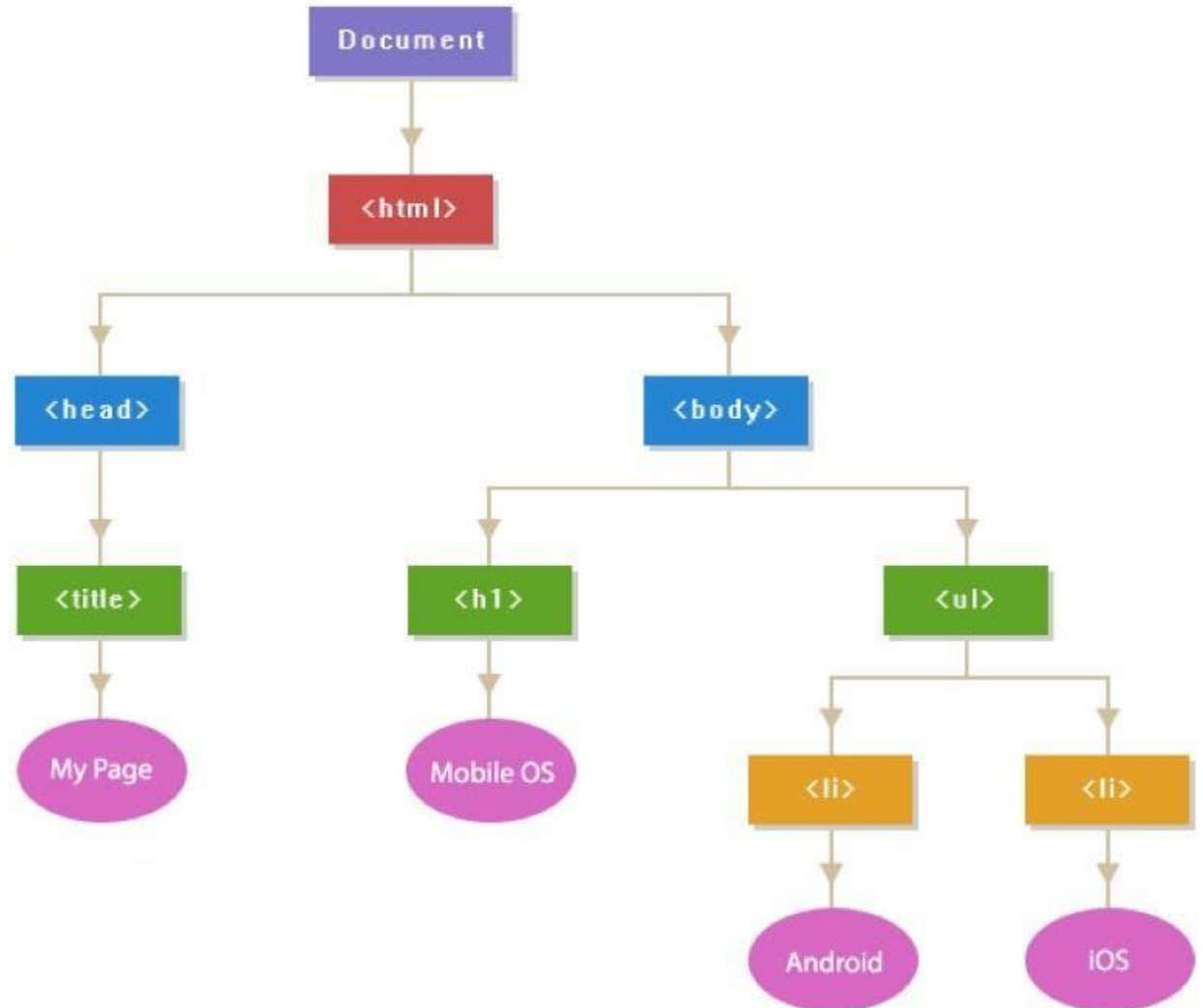
- The DOM is a W3C (World Wide Web Consortium) standard.
- The DOM defines a standard for accessing documents:
  - *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."*
- The way a document content is accessed and modified is called the Document Object Model, or DOM.
- The Objects are organized in a hierarchy.
- This hierarchical structure applies to the organization of objects in a Web document.
  - In the DOM, all parts of the document, such as elements, attributes, text, etc. are organized in a hierarchical tree-like structure; similar to a family tree in real life that consists of parents and children.
  - In DOM terminology these individual parts of the document are known as *nodes*.

# Visualising Elements in DOM Tree

```
<!DOCTYPE html>
<html>
<head>
 <title>My Page</title>
</head>
<body>
 <h1>Mobile OS</h1>

 Android
 iOS

</body>
</html>
```

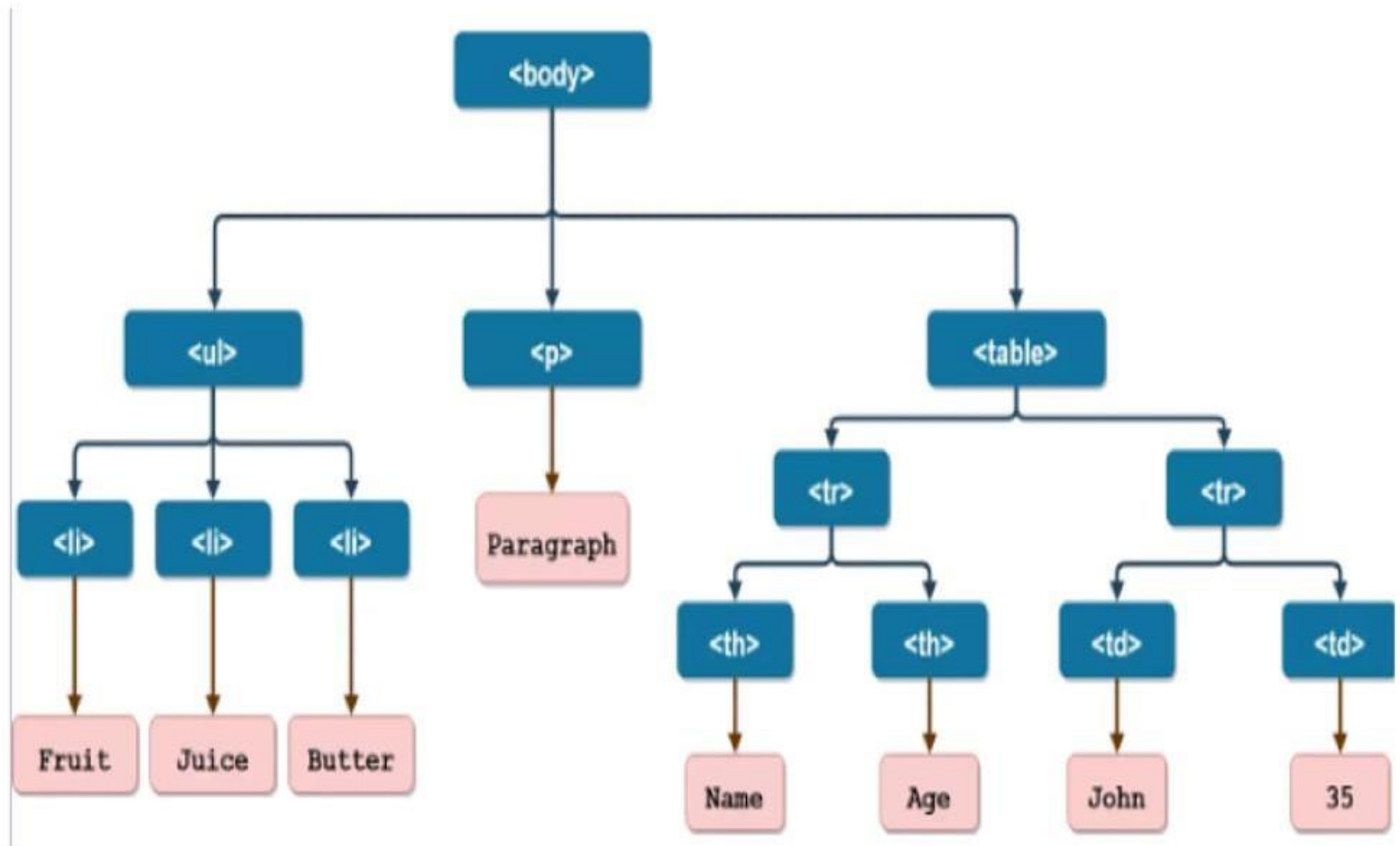


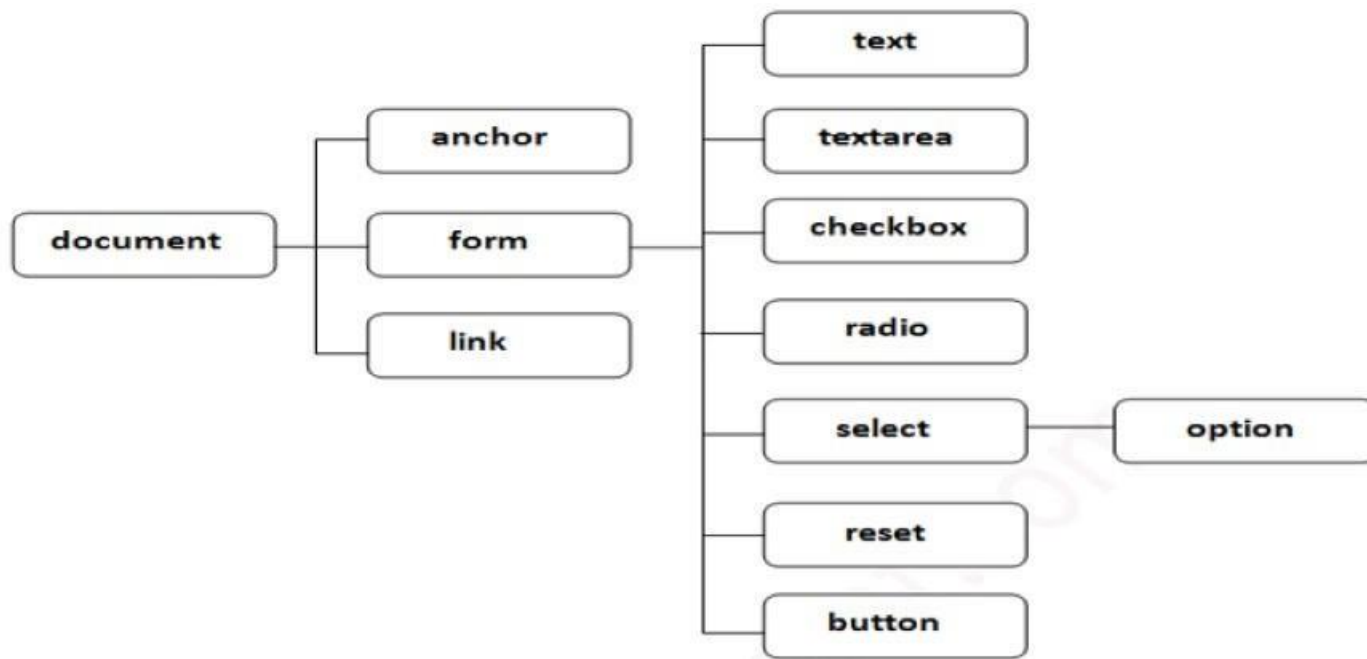
# Visualising Elements in DOM Tree

```
<body>

 Fruit
 Juice
 Butter

<p>Paragraph</p>
<table>
 <tr>
 <th>Name</th>
 <th>Age</th>
 </tr>
 <tr>
 <td>John</td>
 <td>35</td>
 </tr>
</table></body>
```





▪ *The W3C DOM standard is separated into 3 different parts:*

- Core DOM - standard model for **all document types**
- HTML DOM - standard model for **HTML documents**
- XML DOM - standard model for **XML documents**

# What is the HTML DOM?

- The Document Object Model that represents **HTML document** is referred to as HTML DOM.
- HTML DOM which **provides** a standard interface for **accessing and manipulating HTML documents through JavaScript**.
- With the HTML DOM, you can use **JavaScript** to build HTML documents, navigate their **hierarchical structure**, and add, modify, or delete elements and attributes or their content, and so on.
- Almost anything found in an **HTML document can be accessed, changed, deleted, or added using the JavaScript with the help of HTML DOM**.
- Every HTML element in the document is an object.
- The text inside the elements is also an object as well.

# Html DOM

- The DOM provides a way for programming languages, most commonly JavaScript, to interact with and manipulate the content, structure, and style of a web page dynamically.
- It's **not a programming language itself, but rather an API (Application Programming Interface) that allows scripts to access and modify the document.**
- **When a web page is loaded in a browser, the browser creates a DOM representation of that page.**
- This allows developers to use JavaScript to:
  - **Access and select** HTML elements.
  - **Change** the content of elements.
  - **Modify** the structure of the document (add or remove elements).
  - **Change** the style of elements (CSS).
  - **React to events** like button clicks or mouse movements.

# Nodes In HTML DOM ref slide No=79

- In DOM terminology individual element of the document are known as *nodes*.
- The DOM comprises different types of nodes.
- Each node in the DOM tree is identified by a node type.
- Each type of node has a different way to access.
- Element Nodes - Element nodes are themselves called objects in DOM.
  - It comprises most of the DOM which holds other data like text and attributes.
  - Example p, h1, h2, ul, li etc.
  - The element nodes may have other elements nested inside it which are known as children of that element.
- Attribute Nodes - Attribute nodes are attached to the element node and add extra information to the element.
  - Example title attribute, class attribute etc.
- Text Node - Text nodes are contents of the elements and have no children like element nodes.  
Example: `<a href="url">link</a>`

# Nodes In HTML DOM

- From html code demonstrates the **parent/child relationships between the nodes**.
- The topmost node i.e. the **Document node** is the **root node of the DOM tree**, which has **one child**, the `<html>` element. Whereas,
- the `<head>` and `<body>` elements are the **child nodes** of the `<html>` parent node.
- The `<head>` and `<body>` elements are also **siblings** since they are at the same level.
- Further, the **text content inside an element** is a
- . So, for example, "**Mobile OS**" is considered as a **child node** of the `<h1>` that contains it, and so on.
- **Comments** inside the HTML document are nodes in the DOM tree .
- **HTML attributes** such as `id`, `class`, `title`, `style`, etc. are also considered as **nodes in DOM hierarchy** but they don't participate in parent/child relationships like the other nodes do.

```
<!DOCTYPE html>
<html>
<head>
 <title>My Page</title>
</head>
<body>
 <h1>Mobile OS</h1>

 Android
 iOS

</body>
</html>
```

# *JavaScript - HTML DOM Methods*

- HTML DOM **methods** are **actions** you can perform (on HTML Elements).
- HTML DOM **properties** are **values** (of HTML Elements) that you can set or change.

## *The DOM Programming Interface*

- The HTML DOM can be accessed with JavaScript (and with other programming languages).
- In the DOM, all HTML elements are defined as **objects**.
- The programming interface is the properties and methods of each object.
- A **property** is a value that you can get or set (like changing the content of an HTML element).
- A **method** is an action you can do (like add or deleting an HTML element).

# Accessing HTML elements in JavaScript

## Selecting Elements in JavaScript

- JavaScript is most commonly used to get or modify the content or value of the HTML elements on the page, as well as to apply some effects like show, hide, animations etc. But,
- before you can perform any action you need to find or select the target HTML element.
- common ways of selecting the elements on a page and do something with them using the JavaScript.

## Accessing the Topmost Elements

- The topmost elements in an HTML document are available directly as document properties.
- For example, the <html> element can be accessed with document.documentElement property, whereas the <head> element can be accessed with document.head property, and the <body> element can be accessed with document.body property.

# 1.Accessing Elements:

- We can access and change the contents of document by its methods.
- DOM Document methods [for accessing or finding HTML elements](#)

## **Common DOM Methods to access elements**

Method	Description
getElementById()	returns the element having the given id value.
getElementsByName()	returns all the elements having the given name value.
getElementsByTagName()	returns all the elements having the given tag name.
getElementsByClassName()	returns all the elements having the given class name.

# getElementById()

- Selects a single element by its unique id attribute.
- Returns: A single DOM element or null if no element is found.

```
body>
```

```
<p id="p1">Hello</p>
```

```
<script>
```

```
//getElementById implementation
```

```
let p=document.getElementById("p1");
```

```
p.innerHTML="welcome to dom";
```

```
</script>
```

```
</body>
```

# getElementsByClassName(className)

- Selects all elements with the specified class name.
- This method **returns an HTMLCollection (which is like an array) of all elements** that have the specified class name(s).
- syntax: `document.getElementsByClassName("myClass")`

```
<body>
<div class="example">Element 1</div>
<div class="example">Element 2</div>
<button onclick="changeClass()">Highlight All</button>
<script>
//getElementByTagName implementation
function changeClass() {
let collection= document.getElementsByClassName("example");
collection[0].innerHTML="hellow world";
for (let i = 0; i < collection.length; i++) {
 collection[i].style.backgroundColor = "yellow";
} }
</script>
```

## Accessing element

The elements returned by the `getElementsByClassName` method can be accessed by its index values.

# getElementsByTagName(tagName)

- Selects all elements with the specified tag name (e.g., div, p).
- Returns: An HTMLCollection.
- Syntax: **document.getElementsByTagName("tagName");**

```
<body>
 coffe
 Tea
 Milk
<button onclick="makeBold()">Bold Items</button>
</script>
function makeBold() {
 const listItems = document.getElementsByTagName("li");
 for (let item of listItems) {
 item.style.fontWeight = "bold";
 item.style.color = "green";
 }
 </script> </body>
```

**listItems[0].style.backgroundColor = "red";**

# Using Query Selector

**A. `querySelector(cssselector)`::** Returns the first element that matches a CSS selector or null. // '`cssSelector`' can be a **tag name, class, id, or any valid CSS selector**

Returns: A NodeList (array-like, not a true array).

## •Syntax

- `document.querySelector("#myId")`
- `document.querySelector(".myClass")`

`<body>`

`<div class="item">Item 1</div>`

`<div class="item">Item 2</div>`

`<div class="item">Item 3</div>`

`<script>`

`let allItems = document.querySelector(".item");`

`for (let i = 0; i < allItems.length; i++) {`

`allItems[i].style.color = "blue";`

`} </script> </body>`

Item 1

Item 2

Item 3

# Using Query Selector

**B. document.querySelectorAll(**cssselector**):** Similar to `querySelector()`, but it returns of **all elements that match the specified CSS selector**

// '**cssSelector**' can be a **tag name, class, id, or any valid CSS selector**

- Syntax

- `document.querySelectorAll("myId")`
- `document.querySelectorAll("myClass")`

```
<body>
```

```
<div class="item">Item 1</div>
```

```
<div class="item">Item 2</div>
```

```
<div class="item">Item 3</div>
```

```
<script>
```

```
let allItems = document.querySelectorAll("item");
```

```
for (let i = 0; i < allItems.length; i++) {
```

```
 allItems[i].style.fontWeight = "bold";
```

```
 allItems[i].style.color = "green";
```

```
}
```

```
</script>
```

**Item 1**

**Item 2**

**Item 3**

# querySelector vs querySelectorAll

Feature	querySelector	querySelectorAll
Returns	First matching element	All matching elements
Return Type	Single element (or null)	NodeList (can contain 0 or more nodes)
Supports CSS Selectors		
Looping Allowed	 (single element)	 (NodeList can be looped)

# Accessing Form Elements (document.forms)

- Access form fields using document.forms and the name attribute.  
const form = document.forms["formName"]; gets the form.
- const input = form.elements["inputName"]; accesses the input field.

```
<html>
<head>
 <title>Form Access Example</title>
</head>
<body>
 <form name="loginForm">
 <input type="text" name="username"
placeholder="Username">
 <input type="password" name="password"
placeholder="Password">
 <button type="button"
onclick="getFormData()">Submit</button>
 </form>
```

```
<script>
 function getFormData() {
 const form = document.forms["loginForm"];
 const username = form.elements["username"].value;
 const password = form.elements["password"].value;
 alert(`Username: ${username}, Password: ${password}`);
 }
</script>
</body>
</html>
```

## 2.Modifying Elements:

After accessing elements you **can modify and make some changes to the HTML elements.**

- These methods and properties are used to change content, CSS style and get or set attributes to the element.

Properties/Methods	Description
<code>element.innerHTML</code>	This is an element property using which you can get or set the inner content of the element
<code>element.innerText</code>	It returns and sets the inner text of an element to the value given in string, without parsing to HTML
<code>element.style. = value</code>	Using these element properties you can set CSS to the element using javascript
<code>element.getAttribute(attribute_name)</code>	This method is used to get the value of the specified attribute
<code>element.setAttribute(attribute_name, value)</code>	It is the element method that is used to set a new attribute or modify the value of the attribute to the element
<code>element.removeAttribute(attribute_name)</code>	This element method removes the specified attribute from the element

# Javascript - innerHTML

- The **innerHTML** property can be used to **write the dynamic html on the html document**.
- It is used mostly in the web pages to generate the dynamic html such as registration form, comment form, links etc.
- Example : example, we are dynamically writing the html form inside the div name having the id mylocation.
- We are identifying this position by calling the **document.getElementById()** method

```
body>
 <div id="main">
<h1 id="title">Hello World!</h1>
<p id="hint">This is a simple paragraph.</p>
 </div>
<button type="button" onclick="getContents()">Get Contents</button>
<button type="button" onclick="setContents()">Set Contents</button>
 <script>
function getContents() {
 // Getting inner HTML contents
var contents = document.getElementById("main").innerHTML;
alert(contents); // Outputs inner html contents
}
```

```
function setContents() {
// Setting inner HTML contents
 var mainDiv =
document.getElementById("main");
mainDiv.innerHTML = "<p>This is newly
inserted paragraph.</p>";
}
</script>
</body>
```

### 3. create and remove elements

Methods	Description
<code>document.createElement()</code>	Creates an element specified by tag name
<code>document.createTextNode()</code>	Creates a new text node
<code>document.appendChild()</code>	It appends another element to an element in the DOM
<code>document.insertBefore()</code>	This method inserts a node before a specified node, if the node already exists in the DOM then it changes its position
<code>document.replaceChild(new,old)</code>	It replaces an element from DOM with another element
<code>document.removeChild()</code>	It removes an element from DOM
<code>document.remove()</code>	This method removes the node from the DOM it is applied upon
<code>document.write()</code>	It write something in the document

# CSS in JavaScript

---

- One of the many uses of javascript is in **styling HTML document *dynamically***.
- Javascript can add or remove CSS properties to any element either directly or by triggering some event.
- Using javascript you can set almost all CSS properties to the elements like **color, background, border, animation, font style, text alignments, width, height, padding, position** and so on.

## How to Style elements Using Javascript?

- 2 different approaches to alter the style of an element using javascript.
  - a) Directly setting CSS style on element
  - b) Adding or removing class from the element

# 1. JavaScript Set Style (**.style property**)

- Every element in the DOM has a **style object**.
- This style object is also one of the properties of the element.
- The **style** property returns a **CSSStyleDeclaration object**, which represents an element's style attribute.
- To Change style of an elements:
  - Select an element (example - by using `getElementById`)
  - Then access the **style property** of that element.
  - Then set the CSS property of that element using **style property**.
- Syntax: `element.style.css_property = "value";`

```
<body>
 <p id="intro">This is a paragraph.</p>
 <p>This is another paragraph.</p>
 <script>
 // Selecting element
 var elem = document.getElementById("intro");
 // Applying styles on element
 elem.style.color = "blue";
 elem.style.fontSize = "18px";
 elem.style.fontWeight = "bold";
 </script>
</body>
</html>
```

- **Using style.cssText (Multiple Styles at Once):** Set several styles in one string (like writing inline CSS)
- `text.style.cssText = "color: red; font-weight: bold; padding: 10px;";`

# Changing CSS property having hyphen

- There are many CSS properties that have a hyphen in its name, example `background-color`, `border-radius`, `margin-top`, etc and **javascript simply does not accept hyphen in a variable name, function name or in the property name**
- To specify such CSS property simply remove the hyphen from the property name and convert the property in camelcase. Example **'background-color' => backgroundColor**

Property name in CSS	To	in Javascript
background-color	=>	backgroundColor
background-image	=>	backgroundImage
border-bottom	=>	borderBottom
border-top	=>	borderTop
margin-top	=>	marginTop
font-family	=>	fontFamily
and so on...		

# Changing CSS property having hyphen

---

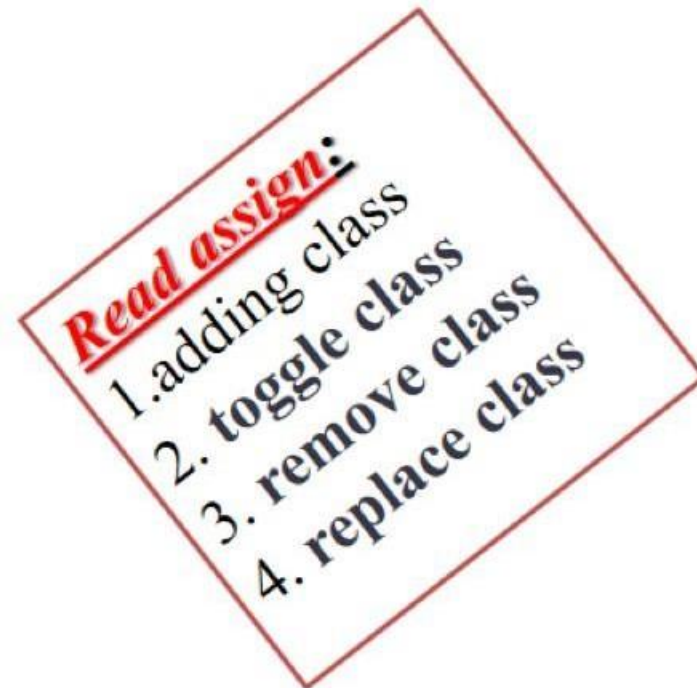
```
const body = document.body;
body.style.backgroundColor = "teal";
```

## **Adding border property to the element**

```
const element =
document.getElementById("para");
element.style.border = "3px";
element.style.borderStyle = "Solid";
element.style.borderColor = "blue";
```

## 2. Javascript Add Class To Element

- Another way to add or remove CSS style from an element is by adding and removing CSS classes from the element.
- In JavaScript, we have className and classList properties to handle classes.
  - **className**: It returns all the classes present in the element in form of string
  - **classList**: It returns DOMTokenList, which contains various methods to handle classes in an element
- Here are few methods to modify classes:
  - classList.add()
  - classList.toggle()
  - classList.remove()
  - classList.replace()



# Events in JavaScript

- An event is an action or a state of any action performed either by the user or by the browser.
- An event can be triggered by the user or by the browser.
- The browser triggers an event when the user interacts with the browser.
- The user triggers an event when the user interacts with the webpage.
- There are many different types of events that can occur in a web browser.
- Some of the most common include:
  - **Mouse Events:**
    - ✓ click: When an element is clicked.
    - ✓ mouseover: When the mouse pointer moves over an element.
    - ✓ mouseout: When the mouse pointer moves off an element.
    - ✓ mousedown: When a mouse button is pressed down over an element.
    - ✓ mouseup: When a mouse button is released over an element.

# Events in JavaScript

## ○ Keyboard Events:

- ✓ **keydown**: When a key is pressed down.
- ✓ **keyup**: When a key is released.
- ✓ **keypress**: When a key is pressed and produces a character value.

## ○ Form Events:

- ✓ **submit**: When a form is submitted.
- ✓ **change**: When the value of an input element changes.
- ✓ **focus**: When an element gains focus.

## ○ Window Events:

- ✓ **load**: When the entire page and all its resources have finished loading.
- ✓ **resize**: When the browser window is resized.
- ✓ **scroll**: When the user scrolls the page.

○ Think of an event like a **doorbell**:

- **Event Trigger**: Someone presses the doorbell (click).
- **Event Listener**: You are waiting inside (JavaScript is listening).
- **Event Handler**: You respond by opening the door (executing a function).
- **Event Target**: The element on which the event occurred.
- **Event Object**: An object that is automatically passed to an event listener function, containing information about the event that occurred.

# Adding Event Listener

---

- Event listeners are something that keeps track of what action to be taken when a specific event occurs.
- It listens to the event and fires the function. but you have to specifically mention in what event what action is to be taken.
- There are 3 different ways by which you can attach an event listener to a certain element:
  - 1) Using HTML attribute
  - 2) Using DOM Property
  - 3) Using addEventListener Method

# 1. inline Event Handler (Using HTML Event Attributes)

- You can directly set an event handler to HTML element as an attribute named **on<event>** and can trigger an event on that action.
- An event handler can be directly attached to an HTML element using attributes that start with on followed by the event name (e.g., **onclick**, **onchange**, **onmouseover**).
- The value of this attribute is either the JavaScript code to execute or a function call.
- When the specified event (e.g., a click) occurs on the element, the associated code or function is triggered, handling the event..
- **Not recommended for larger projects because it mixes HTML with JavaScript**
- **Syntax:** Inline (HTML Attribute): **onclick="handler()"** in HTML.

`<button onclick="alert('Button clicked!')">Click Me</button>`

```
<button onclick="handleClick()">Click Me</button>
<script> function handleClick() {
 alert("Button clicked!");
}
</script>
```

## 2. DOM Property Event Handler

- A DOM Property Event Handler is a **way of assigning an event handler function directly to a property of a DOM element** (like onclick, onmouseover, onchange, etc.).
- How it works
  - ✓ You select the HTML element you want to add an event listener to using DOM manipulation methods (like getElementById, querySelector, etc.).
  - ✓ You access the specific event property on that element's JavaScript object (e.g., element.onclick).
  - ✓ You assign a JavaScript function (either a named function or an anonymous function) to this property.

Syntax: `element.onclick = handler`

```
<button id="myButton">Click
Me</button>
<p id="message"></p>
```

```
<script>
// Get the button element
const button = document.getElementById("myButton");
// Assign a handler to the onclick property
button.onclick = function() {
// This is the event handler
document.getElementById("message").innerHTML = "Button was
clicked!"; };
</script>
```

### 3. addEventListener method(recommended)

- **addEventListener** is a JavaScript method used to attach an event listener to a DOM element, allowing the element to respond to specific events (e.g., clicks, keypresses, mouseovers)
- It takes at least two arguments: the **event type** (e.g., "click") and the **event handler function** to run when the event occurs.
  - **Syntax:** `element.addEventListener(event, handler, [options]);`
- **element:** The DOM element you want to attach the event listener to.
- **event:** A string representing the type of event to listen for (e.g., "click", "mouseover", "keydown", "submit")
- **handlerFunction:** The JavaScript function that you want to execute when the specified event occurs on the element.
- **Event Listener:** is a method (**addEventListener**) used to register a function to be executed when a specific event occurs on a specific DOM element.
- **An event handler:** is the **function** that gets executed when the event occurs. It contains the logic of what should happen.

### 3. addEventListener method

- Example: Button Click Event

```
<button id="clickMe">Click Me!</button>
```

```
<p id="output"></p>
```

// Get the button element

```
const button = document.getElementById("clickMe");
```

// Event Listener: Listens for a "click" event

```
button.addEventListener("click", handleClick); // "handleClick" is the event handler
```

// Event Handler: Defines what happens when the event occurs

```
function handleClick() {
```

```
 document.getElementById("output").textContent = "Button was clicked!";
```

```
}
```

- You can also define the handler directly inside addEventListener:

```
button.addEventListener("click", function() {
```

```
 document.getElementById("output").textContent = "Button was clicked!";
```

```
});
```

# Multiple Listeners for the Same Event

```
<body>
 <button id="myButton">Click Me</button>
<script>
 const button = document.getElementById("myButton");
 // First listener
 button.addEventListener("click", function() {
 alert("First handler!"); });
 // Second listener (does not overwrite the first)
 button.addEventListener("click", function() {
 console.log("Second handler: Button clicked!");
 }); </script></body>
```

- Two listeners are attached to the same "click" event on the button.
- **Both handlers run when the button is clicked:** an alert shows "First handler!", and the console logs "Second handler: Button clicked!".

## One-Time Listener Example:

```
button.addEventListener("click", function ()
{
 alert("You will see this only once!");
}, { once: true });
```

### STEP-BY-STEP SETUP

1. **Create an HTML File:**
2. Open the File in a Web Browser
3. **Open the Developer Console**  
Before clicking the button, open the **Developer Tools**:
  - Press **F12**, or Right-click → "Inspect" → go to the **Console** tab.
4. **Click the Button Now**, click the "Click Me" button.

# Adding Event Listeners for Different Event Types

- you can assign different event listeners to different event types on the same element.
- The following example will assign different event-listener functions to the "click", "mouseover", and "mouseout" events of a button element.

```
<body>
 <button id="myBtn">Click Me</button>
 <script>
 // Selecting button element
 var btn = document.getElementById("myBtn");

 // Defining custom functions
 function sayHello() {
 alert("Hi, how are you doing?");
 }

 function setHoverColor() {
 btn.style.background = "yellow";
 }

 function setNormalColor() {
 btn.style.background = "";
 }

 // Assigning event listeners to the button
 btn.addEventListener("click", sayHello);
 btn.addEventListener("mouseover", setHoverColor);
 btn.addEventListener("mouseout", setNormalColor);
 </script>
</body>
```

# Handling Exception in JavaScript

---

- Sometimes your JavaScript code does not run as smooth as expected, resulting in an error.
- There are a number of reasons that may cause errors, for instance:
  - A problem with network connection
  - A user might have entered an invalid value in a form field
  - Referncing objects or functions that do not exist
  - Incorrect data being sent to or received from the web server
  - A service that the application needs to access might be temporarily unavailable
- JavaScript is a **loosely-typed language**.
- It does **not give compile-time errors**. So some times you will get a runtime error for accessing an undefined variable or calling undefined function etc.

# The try...catch Statement

- JavaScript provides the **try-catch** statement to trap the runtime errors, and handle them gracefully.
- Any code that might possibly throw an error should be placed in the **try block of the statement**, and the code to handle the error is placed in the **catch block**, as shown here:

```
<body> <script>
 try {
 var greet = "Hi, there!";
 document.write(greet);
 // Trying to access a non-existent variable
 document.write(welcome);
 // If error occurred following line won't execute
 alert("All statements are executed successfully.");
 } catch(error) {
 // Handle the error
 alert("Caught error: " + error.message);
 }
 // Continue execution
 document.write("<p>Hello World!</p>");
</script> </body>
```

- JavaScript provides error-handling mechanism to catch runtime errors using **try-catch-finally block**,  
**try** {  
    // Code that may cause an error  
} **catch(error)** {  
    // code to be executed if an error occurs  
}  
**finally**{  
    // code to be executed regardless of an error occurs or not  
}

# throw

- Use throw keyword to raise a custom error.
- throw an error manually by using the throw statement.
- syntax: **throw expression**;

```
<body>
```

```
<script>
```

```
var num = prompt("Please enter an integer value");
```

```
try {
```

```
 if(num == "" || num == null || !Number.isInteger(+num)) {
```

```
 throw new Error("Invalid value!");
```

```
 } else {
```

```
 document.write("Correct value!");
```

```
 }
```

```
} catch(e) {
```

```
 document.write(e.message);
```

```
}
```

```
</script>
```

# Validation Rules

## 1.Name

- Required
- 3–5 characters only
- Letters only (no digits or symbols)

## 2.Email

- Must contain @ and .

## 3.Password + Confirm Password

- At least 6 characters
- Must include: letters, digits, **1 uppercase letter**
- Must match each other

## 4.Age

- Numeric
- Between 18 and 30

## 5.Phone Number

- Must be 10 digits
- Start with 09 or Ethiopian code (+2519)

## Sign Up

Name:

Email:

Password:

Confirm

Password:

Age:

Phone Number:

# JavaScript Form Validation

- Validation is the process of **checking** data against a **standard** or **requirement**.
- **Form validation** normally used to occur at the server, after client entered necessary data and then pressed the Submit button.
- If the **data entered by a client was incorrect** or was simply missing, the **server** would have to send all the data back to the client and request that the form be resubmitted with correct information.
- This was really a lengthy process which used to put a lot of burden **on the server**.
- **JavaScript** provides a way to validate form's data on the client's computer before sending it to the web server.
- It is important to validate the form submitted by the user because it can have inappropriate values. So validation is must.
- Through JavaScript, we can **validate name, password, email, date, mobile number etc fields**.

# JavaScript Form Validation

---

- The form validation process typically consists of **two parts**—
  1. **required fields validation** which is performed to make sure that all the mandatory fields are filled in (Emptiness, Confirm password, Length validation etc)
  2. the **data format validation** which is performed to ensure that the type and format of the data entered in the form is valid. (email validation, mobile number validation, etc )

# JavaScript Form Validation

---

- **Step for Form Processing Using JavaScript**
  1. Create the HTML Form
  2. Add Event Listeners for Form Submission
  3. Retrieve Form Data
  4. Validate Input Data
  5. Provide Feedback to the User
  6. Submit Data to the Server (Optional)

- If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

```
function validateForm() {
 let x = document.forms["myForm"]["fname"].value;
 if (x == "") {
 alert("Name must be filled out");
 return false;
 }
}
```

The function can be called when the form is submitted:

### HTML Form Example

```
<form name="myForm" action="/action_page.php" onsubmit="return
validateForm()" method="post">
```

```
Name: <input type="text" name="fname">
```

```
<input type="submit" value="Submit">
```

```
</form>
```

# Data Validation

- ❖ Data validation is the process of ensuring that user input is clean, correct, and useful.
- Typical validation tasks are:
  - has the user filled in all required fields?
  - has the user entered a valid date?
  - has the user entered text in a numeric field?
- ✓ Most often, the purpose of data validation is to ensure correct user input.
- ✓ Validation can be defined by many different methods, and deployed in many different ways.
- *Server side validation* is performed by a web server, after input has been sent to the server.
- *Client side validation* is performed by a web browser, before input is sent to a web server.

# HTML Constraint Validation

- HTML5 introduced a new HTML validation concept called **constraint validation**.
- HTML constraint validation is based on:
  - Constraint validation **HTML Input Attributes**
  - Constraint validation **CSS Pseudo Selectors**
  - Constraint validation **DOM Properties and Methods**

## *Constraint Validation HTML Input Attributes*

Attribute	usage
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

# Constraint Validation CSS Pseudo Selectors

Selector	usage
:disabled	Selects input elements with the "disabled" attribute specified
:invalid	Selects input elements with invalid values
:optional	Selects input elements with no "required" attribute specified
:required	Selects input elements with the "required" attribute specified
:valid	Selects input elements with valid values

## Constraint Validation DOM Methods

Property	usage
checkValidity()	Returns true if an input element contains valid data.
setCustomValidity()	Sets the validationMessage property of an input element.

## The checkValidity() Method

```
<input id="id1" type="number" min="100" max="300">
<button onclick="myFunction()">OK</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function myFunction() {
 var inpObj = document.getElementById("id1");
 if (inpObj.checkValidity() == false) {
 document.getElementById("demo").innerHTML = inpObj.validationMessage;
 }
}
```

```
</script>
```

## *Constraint Validation DOM Properties*

Property	Description
validity	Contains boolean properties related to the validity of an input element.
validationMessage	Contains the message a browser will display when the validity is false.
willValidate	Indicates if an input element will be validated.

### **Validity Properties**

Property	Description
customError	Set to true, if a custom validity message is set.
patternMismatch	Set to true, if an element's value does not match its pattern attribute.
rangeOverflow	Set to true, if an element's value is greater than its max attribute.
rangeUnderflow	Set to true, if an element's value is less than its min attribute.
stepMismatch	Set to true, if an element's value is invalid per its step attribute.
tooLong	Set to true, if an element's value exceeds its maxLength attribute.
typeMismatch	Set to true, if an element's value is invalid per its type attribute.
valueMissing	Set to true, if an element (with a required attribute) has no value.
valid	Set to true, if an element's value is valid.

# Automatic HTML Form Validation

- HTML form validation can be performed **automatically by the browser**:
- If a form field (fname) is empty, the **required attribute** prevents this form from being submitted:

```
<body>
```

```
<h2>JavaScript Validation</h2>
```

```
<form action="/action_page.php" method="post">
```

```
 <input type="text" name="fname" required>
```

```
 <input type="submit" value="Submit">
```

```
</form>
```

```
<p>If you click submit, without filling out the text field,
your browser will display an error message.</p>
```

```
</body></html>
```

## The min(The rangeOverflow Property) and max(The rangeUnderflow Property) Attributes

- The input min and max attributes specify the minimum and maximum values for an input field.
- The min and max attributes work with the following input types: [number](#), [range](#), [date](#), [datetime-local](#), [month](#), [time](#) and [week](#).
- Use the max and min attributes together to create a range of legal values.

```
<body> <h1>The input min and max attributes</h1>
```

```
<p>The min and max attributes specify the minimum and maximum values for an input element.</p>
```

```
<form action="/action_page.php">
```

```
 <label for="datemax">Enter a date before 1980-01-01:</label>
```

```
 <input type="date" id="datemax" name="datemax" max="1979-12-31">


```

```
 <label for="datemin">Enter a date after 2000-01-01:</label>
```

```
 <input type="date" id="datemin" name="datemin" min="2000-01-02">


```

```
 <label for="quantity">Quantity (between 1 and 5):</label>
```

```
 <input type="number" id="quantity" name="quantity" min="1" max="5">


```

```
 <input type="submit" value="Submit">
```

```
</form></body></html>
```

## Example, validate the name and password.

The name can't be empty and password can't be less than 6 characters long.

**<script>**

```
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;

if (name==null || name==""){
 alert("Name can't be blank");
 return false;
}else if(password.length<6){
 alert("Password must be at least 6 characters long.");
 return false;
}
}
```

**</script>**

**<body>**

**<form** name="myform" method="post" action="abc.jsp" onsubmit="return validateform()" **>**

Name: **<input** type="text" name="name">**><br/>**

Password: **<input** type="password" name="password">**><br/>**

**<input** type="submit" value="register"**>**

**</form>**

# JavaScript Number Validation

- Let's validate the **textfield** for numeric value only.
- Here, we are using **isNaN()** function.

```
<script>
```

```
function validate(){
var num=document.myform.num.value;
if (isNaN(num)){
 document.getElementById("numloc").innerHTML="Enter Numeric value only";
 return false;
}else{
 return true;
}
}
```

```
</script>
```

```
<form name="myform" onsubmit="return validate()" >
Number: <input type="text" name="num">

<input type="submit" value="submit">
</form>
```

# Javascript Regular Expression

---

- Regular Expressions, commonly known as "**regex**" or "**RegExp**", are a specially formatted text strings **used to find patterns in text**.
- For example, it can be used to verify whether the format of data i.e. **name, email, phone number, etc.** entered by the user is correct or not, find or replace matching string within text content, and so on.
- **Defining Regular Expressions**
- In JavaScript, regular expressions are represented by **RegExp object**, which is a native JavaScript object like String, Array, and so on.
- There are **two ways of creating a new RegExp object**
  - **literal syntax** and **RegExp() constructor**.
- **The literal syntax** uses forward slashes (/pattern/) to wrap the regular expression pattern.
- **The constructor syntax** uses quotes ("pattern").
- Example: demonstrates both ways of creating a regular expression that matches any string that begins with "Mr."

```
<body>
 <script>
 var str = "Mr. Gray";

 // Literal syntax
 var regex1 = /^Mr\./;

 if(regex1.test(str)) {
 document.write("Match found!");
 } else {
 document.write("Match not
found.");
 }
 </script>
</body>
```

```
document.write("<hr>");

// Constructor syntax
var regex2 = new RegExp("^Mr\\.");

if(regex2.test(str)) {
 document.write("Match found!");
} else {
 document.write("Match not
found.");
}
</script>
</body>
```

# Pattern Matching with Regular Expression

---

- Regular expression patterns include the use of letters, digits, punctuation marks, etc., plus a set of special regular expression characters
- The characters that are given special meaning within a regular expression, are: . \* ? + [ ] ( ) { } ^ \$ | \.
- You will need to backslash these characters whenever you want to use them literally.
- For example, if you want to match ".", you'd have to write \.
- All other characters automatically assume their literal meanings.

# Character Classes

---

- Square brackets surrounding a pattern of characters are called a **character class** e.g. [abc].
- A **character class** always matches a single character out of a list of specified characters that means the expression [abc] matches only a, b or c character.
- **Negated character classes** can also be defined that match any character except those contained within the brackets.
- A negated character class is defined by placing a caret (^) symbol immediately after the opening bracket, like [^abc], which matches any character except a, b, and c.
- **define a range of characters** by using the **hyphen (-) character inside a character class**, like [0-9]. Let's look at some examples of the character classes:

## some examples of the character classes:

RegExp	What it Does
[abc]	Matches any one of the characters a, b, or c.
[^abc]	Matches any one character other than a, b, or c.
[a-z]	Matches any one character from lowercase a to lowercase z.
[A-Z]	Matches any one character from uppercase a to uppercase z.
[a-Z]	Matches any one character from lowercase a to uppercase Z.
[0-9]	Matches a single digit between 0 and 9.
[a-z0-9]	Matches a single character between a and z or between 0 and 9.

# Predefined Character Classes

- Some character classes such as digits, letters, and whitespaces are used so frequently that there are shortcut names for them.
- The following table lists those predefined character classes:

Shortcut	What it Does
.	Matches any single character except newline <code>\n</code> .
<code>\d</code>	matches any digit character. Same as <code>[0-9]</code>
<code>\D</code>	Matches any non-digit character. Same as <code>[^0-9]</code>
<code>\s</code>	Matches any whitespace character (space, tab, newline or carriage return character). Same as <code>[ \t\n\r]</code>
<code>\S</code>	Matches any non-whitespace character. Same as <code>[^ \t\n\r]</code>
<code>\w</code>	Matches any word character (defined as a to z, A to Z, 0 to 9, and the underscore). Same as <code>[a-zA-Z_0-9]</code>
<code>\W</code>	Matches any non-word character. Same as <code>[^a-zA-Z_0-9]</code>

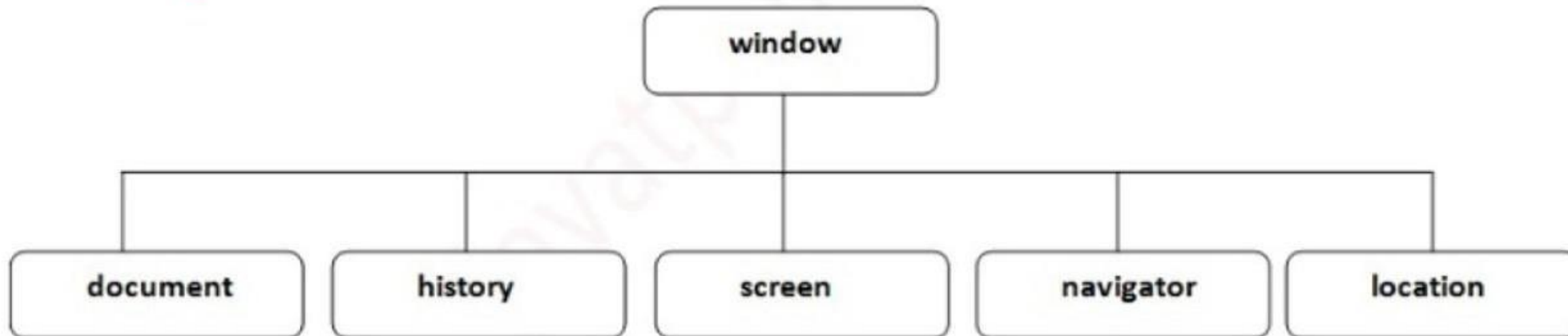
# Repetition Quantifiers

- In the previous section we've learnt how to match a single character in a variety of fashions. But what if you want to match on more than one character?
- For example, let's say you want to find out words containing one or more instances of the letter p, or words containing at least two p's, and so on.
- This is where quantifiers come into play. With quantifiers you can specify how many times a character in a regular expression should match.
- The following table lists the various ways to quantify a particular

RegExp	What it Does
p+	Matches one or more occurrences of the letter p.
p*	Matches zero or more occurrences of the letter p.
p?	Matches zero or one occurrences of the letter p.
p{2}	Matches exactly two occurrences of the letter p.
p{2,3}	Matches at least two occurrences of the letter p, but not more than three occurrences.
p{2,}	Matches two or more occurrences of the letter p.
p{,3}	Matches at most three occurrences of the letter p

# Browser Object Model (BOM)

- The **Browser Object Model** (BOM) is used to interact with the browser.
- the BOM deals with the browser and the environment in which the document is rendered.
- The Browser Object Model (BOM) allows JavaScript to "talk to" the browser.
- The default object of browser is window means you can call all the functions of window by specifying window or directly.
- For example: `window.alert("hello student");` or `alert("hello student");`
- The BOM provides objects like window, navigator, screen, location, and history.



# window object

---

- The **window object** represents a window in browser.
- An object of window is created automatically by the browser.
- Window is the object of browser, **it is not the object of javascript.**
- All global JavaScript objects, functions, and variables automatically become members of the window object.
- Global variables are properties of the window object.
- Global functions are methods of the window object.
- Even the document object (of the HTML DOM) is a property of the window object:
  - `window.document.getElementById("header");`
  - is the same as: `document.getElementById("header");`
- **Methods of window object**
- `alert()` ,`confirm()` ,`prompt()` ,`open()`,`close()`,`setTimeout()`

# Pop up Boxes

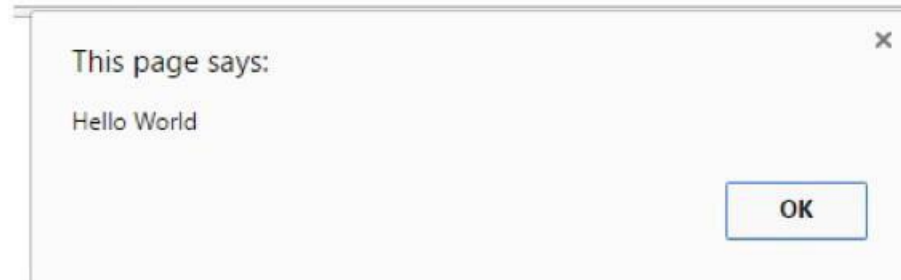
- Popup boxes can be used to raise an alert, or to get confirmation on any input or to have a kind of input from the users.
- JavaScript supports **three** types of popup boxes.
  - Alert box
  - Confirm box
  - Prompt box

## Alert Box

- An **alert box** is used if you want to **make sure** information **comes through** to the **user**.
- When an alert box pops up, the user will **have to click "OK"** to proceed.
- It can be used to display the result of validation.

### Code

```
<html>
 <head>
 <title>Alert Box</title>
 </head>
 <body>
 <script>
 alert("Hello World");
 </script>
 </body>
</html>
```

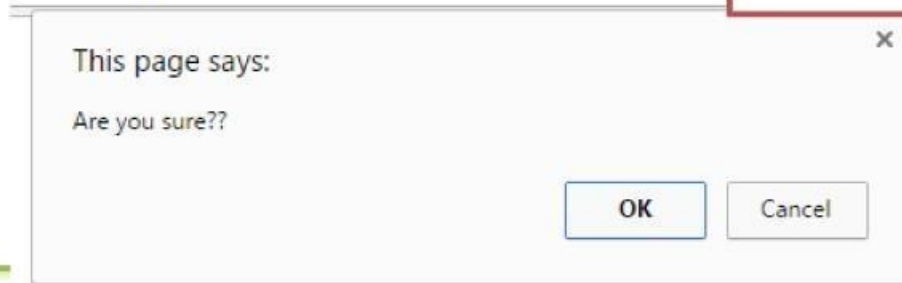


## Confirm Box

- A **confirm box** is used if you want the user to **accept something**.
- **When** a confirm box **pops up**, the user will have to **click** either "**OK**" or "**Cancel**" to proceed, If the user clicks "**OK**", the box **returns true**. If the user clicks "**Cancel**", the box **returns false**.
- Example :

### Code

```
<script>
var a = confirm("Are you sure??");
if(a==true) {
 alert("User Accepted");
}
else {
 alert("User Canceled");
}
</script>
```

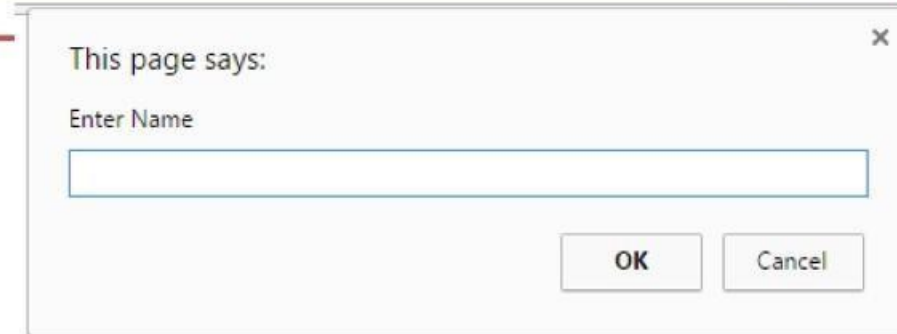


## Prompt Box

- A **prompt box** is used if you want the user to **input a value**.
- **When** a prompt box **pops up**, user have to click either "**OK**" or "**Cancel**" to proceed, If the user clicks "**OK**" the box **returns the input value**, If the user clicks "**Cancel**" the box **returns null**.

### Code

```
<script>
var a = prompt("Enter Name");
alert("User Entered " + a);
</script>
```



# Window Size

---

- Two properties can be used to determine the size of the browser window.
- Both properties return the sizes in pixels:
  - `window.innerHeight` - the inner height of the browser window (in pixels)
  - `window.innerWidth` - the inner width of the browser window (in pixels)
- **Some other methods:**
  - `window.open()` - open a new window
  - `window.close()` - close the current window
  - `window.moveTo()` -move the current window
  - `window.resizeTo()` -resize the current window

# Window Screen

The **JavaScript screen object** holds information of browser screen.

The `window.screen` object contains information about the user's screen such as resolution (i.e. width and height of the screen), color depth, pixel depth, etc.

The following table shows the properties of the `screen` object.

Property	Description
width	returns the width of the screen
height	returns the height of the screen
availWidth	returns the available width
availHeight	returns the available height
colorDepth	returns the color depth
pixelDepth	returns the pixel depth.

## *Window Screen Width and height*

The `screen.width` property returns the width of the visitor's screen in pixels.

## *Example*

Display the width of the screen in pixels:

```
<script>
```

```
document.writeln("
screen.width: "+screen.width);
document.writeln("
screen.height: "+screen.height);
document.writeln("
screen.availWidth: "+screen.availWidth);
document.writeln("
screen.availHeight: "+screen.availHeight);
document.writeln("
screen.colorDepth: "+screen.colorDepth);
document.writeln("
screen.pixelDepth: "+screen.pixelDepth);
```

- Full URL: <https://www.example.com:8080/path/to/page?query=1#section>  
Protocol: https:
- Host: [www.example.com:8080](https://www.example.com:8080)
- Hostname: [www.example.com](https://www.example.com)
- Port: 8080 Pathname: /path/to/page
- Search: ?query=1
- Hash: #section

# JavaScript Location

- If you want to get all information about the current page URL, you can use the **location** object.
- It is a global object that is available in all browsers. It is used to get the current URL, hostname, pathname, protocol, port, and other information.
- To access the location object, you can use the **window.location** or **document.location** or just **location**.
- Here is the list of all properties of the location object.

Property	Description
<b>location.href</b>	Returns the entire URL of the current page.
<b>location.protocol</b>	Returns the protocol of the current page URL. It returns the protocol with a colon (:) at the end. For example, http: or https:
<b>location.host</b>	Returns the hostname and port number of the current page URL.
<b>location.hostname</b>	Returns the hostname of the current page URL.
<b>location.port</b>	Returns the port number of the current page URL.
<b>location.pathname</b>	Returns the path of the current page URL.
<b>location.search</b>	Returns the query string of the current page URL.

## Example:

```
<body>
<script>
// get current URL path
document.write(window.location.href + "
");
 // Prints protocol like http: or https:
document.write(window.location.protocol + "
");

// Prints hostname with port like localhost or
localhost:3000
document.write(window.location.host + "
");

// Prints hostname like localhost or www.example.com
document.write(window.location.hostname + "
");
```

```
// Prints port number like 3000
document.write(window.location.port + "
");

// Prints pathname like /products/search.php
document.write(window.location.pathname + "
");

// Prints query string like ?q=ipad
document.write(window.location.search + "
");

</script>
<p>Note: If the URL does not
contain a specific component (e.g., port number, and
fragment identifier here), it will be set to "</p>
</body>
```

- Full URL: <https://www.example.com:8080/path/to/page?query=1#section>
- Protocol: [https](https://www.example.com:8080/path/to/page?query=1#section):
- Host: [www.example.com:8080](https://www.example.com:8080/path/to/page?query=1#section)
- Hostname: [www.example.com](https://www.example.com:8080/path/to/page?query=1#section)
- Port: 8080 Pathname: [/path/to/page](https://www.example.com:8080/path/to/page?query=1#section)
- Search: [?query=1](https://www.example.com:8080/path/to/page?query=1#section)
- Hash: [#section](https://www.example.com:8080/path/to/page?query=1#section)

# Methods of Location Object

Read Assign: Window History object & Window Navigator object

- The location object has many methods. These methods are used to change the current page URL.
- Here is the list of all methods of the location object.

Method	Description
<code>location.assign()</code>	Loads a new document.
<code>location.reload()</code>	Reloads the current document.
<code>location.replace()</code>	Replaces the current document with a new one.

- Using the `assign()` and `replace()` method of location object to load another document by passing the address of the new document.
- To load new documents, pass the URL as a parameter.
- Example: `location.assign("https://www.tutorial.com")`.
- There is another method `replace()` which does the same task of loading new documents in the window but it does not create any entry of browser history, hence can't go to the previous page after using this but can go to the forward direction.
- Example: `location.replace("https://www.tutorial.com");`

# Navigator Object:

- The navigator object provides information about the browser and the operating system, such as
  - `navigator.userAgent`,
  - `navigator.platform`, and
  - `navigator.language`.

# History Object:

- The history object provides access to the browser's session history, allowing you to navigate back and forth through the visited pages using
  - `history.back()` and
  - `history.forward()`.

# JavaScript Cookies

- Cookies are small pieces of data stored in small text files on the client-side by the web browser.
- They are often used to remember information about the user, such as login details or preferences, interests etc .
- A cookie is basically a string of name-value pairs separated by semicolons.
  - *E.g.* “color = red; expires = fri, 5 August 2024 01:00:00 UTC;”
- When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user.
- Cookies were invented to solve the problem "how to remember information about the user":
  - when a visits any webpage then user's name can be stored in cookie
  - Next time when user visit the page then cookie remembers the name

# *Create ,read change and delete a Cookie with JavaScript*

- JavaScript can create, read, and delete cookies with the *document.cookie* property.
- With JavaScript, a cookie can be created like this:
  - `document.cookie = "username=Abi Kebi";`
- You can also add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:
  - `document.cookie = "username= Abi Kebi; expires=Thu, 18 Dec 2013 12:00:00 UTC";`
- With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.
  - `document.cookie = "username= Abi Kebi; expires=Thu, 18 Dec 2013 12:00:00 UTC; path="/;`
- With JavaScript, cookies can be read like this:
  - `var x = document.cookie;`
- With JavaScript, you can change a cookie the same way as you create it:
  - `document.cookie="username=John Smith; expires=Thu, 18 Dec 2013 12:00:00 UTC; path="/;`
- Deleting a cookie is very simple. Just set the expires parameter to a passed date:
  - `document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC";`

## *Example :How Cookie works*

- we will create a cookie that stores the name of a visitor.
- The first time a visitor arrives to the web page, he/she will be asked to fill in his/her name. The name is then stored in a cookie.
- The next time the visitor arrives at the same page, he/she will get a welcome message.
- For this scenario we will create 3 JavaScript functions:
  - ✓ A function to set a cookie value
  - ✓ A function to get a cookie value
  - ✓ A function to check a cookie value

## *A Function to Set a Cookie*

- First, we create a function that stores the name of the visitor in a cookie variable:

```
function setCookie(cname, cvalue, exdays) {
 const d = new Date();
 d.setTime(d.getTime() + (exdays*24*60*60*1000));
 let expires = "expires=" + d.toUTCString();
 document.cookie = cname + "=" + cvalue + ";" + expires + ";path=/";
}
```

- **Here**

- name of the cookie (cname),
- the value of the cookie (cvalue), and
- the number of days until the cookie should expire (exdays).

## *A Function to Get a Cookie*

- Then, we create a function that **returns the value of a specified cookie**:

```
function getCookie(cname) {
 let name = cname + "=";
 let decodedCookie = decodeURIComponent(document.cookie);
 let ca = decodedCookie.split(';');
 for(let i = 0; i <ca.length; i++) {
 let c = ca[i];
 while (c.charAt(0) == ' ') {
 c = c.substring(1);
 }
 if (c.indexOf(name) == 0) {
 return c.substring(name.length, c.length);
 }
 }
 return "";
}
```

## *Function explained:*

- Take the cookiename as parameter (cname).
- Create a variable (name) with the text to search for (cname + "=").
- Decode the cookie string, to handle cookies with special characters, e.g. '\$'
- Split document.cookie on semicolons into an array called ca (ca = decodedCookie.split(';')).
- Loop through the ca array (i = 0; i < ca.length; i++), and read out each value c = ca[i]).
- If the cookie is found (c.indexOf(name) == 0), return the value of the cookie  
(c.substring(name.length, c.length)).
- If the cookie is not found, return "".

## *A Function to Check a Cookie*

- Last, we create the function that checks if a cookie is set.
- If the cookie is set it will display a greeting.
- If the cookie is not set, it will display a prompt box, asking for the name of the user, and stores the username cookie for 365 days, by calling the setCookie function:

```
function checkCookie() {
 let username = getCookie("username");
 if (username != "") {
 alert("Welcome again " + username);
 } else {
 username = prompt("Please enter your name:", "");
 if (username != "" && username != null) {
 setCookie("username", username, 365);
 }
 }
}
```

## All Together Now

```
function setCookie(cname, cvalue, exdays) {
 const d = new Date();
 d.setTime(d.getTime() + (exdays * 24 * 60 * 60 * 1000));
 let expires = "expires=" + d.toUTCString();
 document.cookie = cname + "=" + cvalue + ";" + expires
+ ";path=/";
}
function getCookie(cname) {
 let name = cname + "=";
 let ca = document.cookie.split(';');
 for(let i = 0; i < ca.length; i++) {
 let c = ca[i];
 while (c.charAt(0) == ' ') {
 c = c.substring(1);
 }
 if (c.indexOf(name) == 0) {
 return c.substring(name.length, c.length);
 }
 }
 return "";
}
```

```
function checkCookie() {
 let user = getCookie("username");
 if (user != "") {
 alert("Welcome again " + user);
 } else {
 user = prompt("Please enter your name:", "");
 if (user != "" && user != null) {
 setCookie("username", user, 365);
 }
 }
}
```

**THANK YOU**

Programm